The East Kalyan Welfare Society's
MODEL COLLEGE OF SCIENCE AND COMMERCE
(Affiliated to Mumbai University)
CHINCHPADA ROAD, RAJBHAR NAGAR, KALYAN (E)

INDEX

Sr No.	Title	Date	Sign
	Module-1		
1	Write the program for the following: a. Create an MVC application to perform basic Arithmetic Operations.	**19/07/26**	
	b. Create an application to print Floyd's triangle till n rows in C#.	**19/07/26**	
	c. Write C# code to arrange the name of cities in sorted order. Accept name of 10 cities from the user	**24/07/26**	
2	Write the program for the following: a. Function Overloading	**24/06/26**	
	b. Inheritance (all types)	**29/06/26**	
	c. Constructor overloading	**29/06/26**	
	d. Interfaces	**01/07/26**	
	e. Using Delegates and events	**01/07/26**	
	f. Exception handling	**03/07/26**	
3	Write the program for the following: a. Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.	**08/07/26**	
	b. Write a program using multicast delegate.	**10/07/26**	
	c. Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).	**15/07/26**	
4	Create an ASP.NET Web Forms application using simple server controls. a. Design a Student Registration Form using the following ASP.NET controls:	**22/07/26**	
	b. Create a Registration form to demonstrate use of various Validation controls.	**24/07/26**	
	c. Create Web Form to demonstrate use of Adrotator Control.	**31/07/26**	

5	Create the following application: a. Create a web application to demonstrate the use of different types of Cookies.	**05/08/26**	
	b. Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.	**12/08/26**	
	Module-2		
1.	a. Create a simple website for a College Information Portal and convert it into a Progressive Web Application.	**14/08/26**	
	b.Create a PWA that displays a custom offline page when the user is not connected to the internet.	**19/08/26**	
2.	a. Develop a PWA that works in offline mode using Service Worker.	**21/08/26**	
	b. Create a Student Record Management PWA.	**27/08/26**	
3	a. Create an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json. This is an important file that would be created in the next step.	**07/09/26**	
	b. Create a simple Web API that returns Student data in JSON format.	**09/09/26**	
4	a. Create a PWA with a Web App Manifest file including: App name Icons Start URL Display mode Implement Push Notifications in a PWA.	**21/09/26**	
	b. Develop an E-Commerce PWA with product listing and offline cart functionality.	**24/09/26**	

Module -1

Practical 1a: MVC Application for Basic Arithmetic Operations

AIM To create an ASP.NET Core MVC application that performs basic arithmetic operations (Addition, Subtraction, Multiplication, Division) using C#.

Step 1:

1.Create Project Open Visual Studio → File → New → Project
2. Select 'ASP.NET Core Web App (Model-View-Controller)'
3. Name the project: ArithmeticMVC → Click Create
4. Select Framework: .NET 6.0 (or .NET 8.0) → Click Create

Step 2:

Create the Model Right-click Models folder → Add → Class → Name: ArithmeticModel.cs

Program Code

Models/ArithmeticModel.cs

```
using System.ComponentModel.DataAnnotations;
 namespace ArithmeticMVC.Models {
   public class ArithmeticModel
      {
        [Required]
        [Display(Name = "First Number")] public double Number1 { get; set; }
        [Required]
        [Display(Name = "Second Number")] public double Number2 { get; set; }
        public string Operation { get; set; } = "";
        public double Result { get; set; }
        public string ErrorMessage { get; set; } = "";
      }
  }
```

Controllers/HomeController.cs

```
using ArithmeticMVC.Models;
using Microsoft.AspNetCore.Mvc;

namespace ArithmeticMVC.Controllers
{
   public class HomeController : Controller
```

```
    {
        public IActionResult Index()
        {
            return View(new ArithmeticModel());
        }

        [HttpPost]
        public IActionResult Index(ArithmeticModel model)
        {
            if (ModelState.IsValid)
            {
                switch (model.Operation)
                {
                    case "Add":
                        model.Result = model.Number1 + model.Number2;
                        break;
                    case "Subtract":
                        model.Result = model.Number1 - model.Number2;
                        break;
                    case "Multiply":
                        model.Result = model.Number1 * model.Number2;
                        break;
                    case "Divide":
                        if (model.Number2 == 0)
                        {
                            model.ErrorMessage = "Error: Division by zero!";
                        }
                        else
                        {
                            model.Result = model.Number1 / model.Number2;
                        }
                        break;
                }
            }

            return View(model);
        }
    }
}
```

Views/Home/Index.cshtml

```
@model ArithmeticMVC.Models.ArithmeticModel
@{
   ViewData["Title"] = "Arithmetic Operations";
}

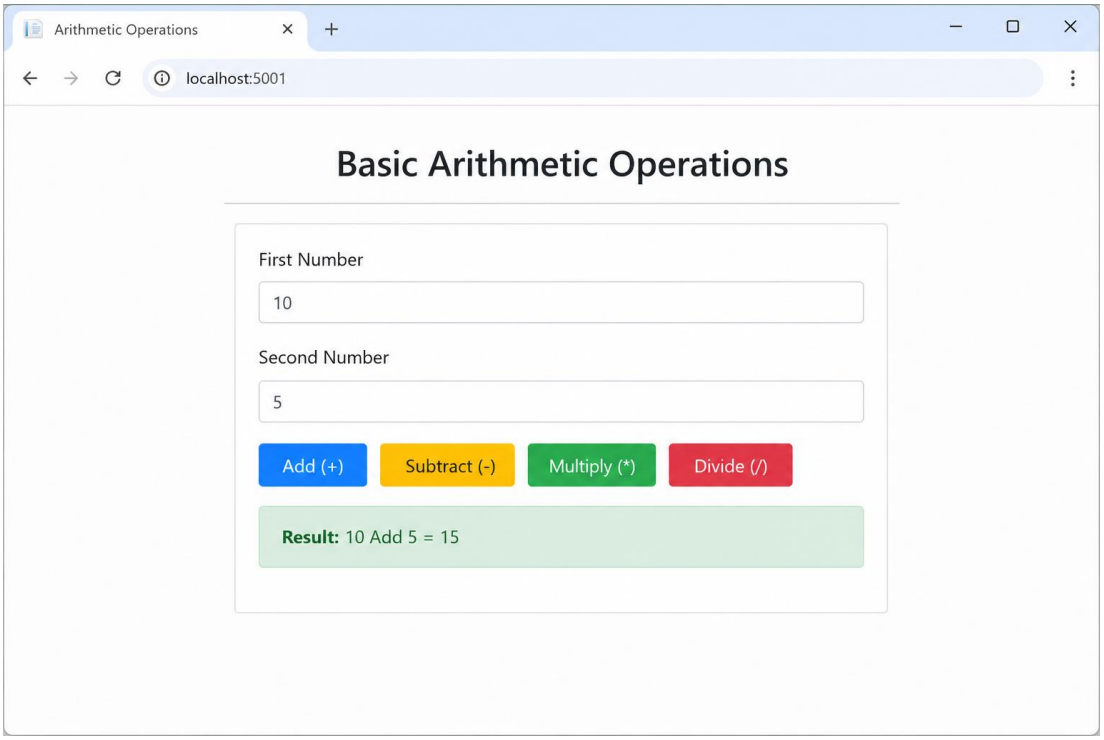
<h2 class="text-center mt-3">Basic Arithmetic Operations</h2>
<hr />

<div class="container">
   <form asp-action="Index" method="post">
      <div class="row justify-content-center">
         <div class="col-md-6">
            <div class="mb-3">
               <label asp-for="Number1" class="form-label"></label>
               <input asp-for="Number1" class="form-control" />
               <span asp-validation-for="Number1" class="text-danger"></span>
            </div>
            <div class="mb-3">
               <label asp-for="Number2" class="form-label"></label>
               <input asp-for="Number2" class="form-control" />
               <span asp-validation-for="Number2" class="text-danger"></span>
            </div>
            <div class="mb-3">
               <button type="submit" name="Operation" value="Add" class="btn btn-
primary">Add (+)</button>
               <button type="submit" name="Operation" value="Subtract" class="btn btn-
warning">Subtract (-)</button>
               <button type="submit" name="Operation" value="Multiply" class="btn btn-
success">Multiply (*)</button>
               <button type="submit" name="Operation" value="Divide" class="btn btn-
danger">Divide (/)</button>
            </div>

            @if (Model.ErrorMessage != "")
            {
               <div class="alert alert-danger">@Model.ErrorMessage</div>
            }
            else if (Model.Operation != "")
```

```
            {
                <div class="alert alert-success">
                    <strong>Result:</strong> @Model.Number1 @Model.Operation
@Model.Number2 = @Model.Result
                </div>
            }
        </div>
    </div>
  </form>
</div>
```

Output:



Practical 1b: Floyd's Triangle till N Rows in C#

AIM: To write a C# program to print Floyd's triangle up to N rows, where N is input by the user.

Steps to Follow:

1. Open Visual Studio → File → New → Project
2. Select 'Console App' → Name: FloydTriangle → Click Create
3. Write the following code in Program.cs
4. Press Ctrl+F5 to run the application

Program.cs

```
using System;

namespace FloydTriangle
{
   class Program
   {
      static void Main(string[] args)
      {
         Console.WriteLine("=== Floyd's Triangle ===");
         Console.Write("Enter the number of rows: ");
         int n = int.Parse(Console.ReadLine());

         int number = 1;

         Console.WriteLine("\nFloyd's Triangle:");
         Console.WriteLine(new string('-', 30));

         for (int i = 1; i <= n; i++)
         {
            for (int j = 1; j <= i; j++)
            {
               Console.Write(number + "\t");
               number++;
            }
            Console.WriteLine();
         }

         Console.WriteLine(new string('-', 30));
```

```
            Console.WriteLine($"Total numbers printed: {number - 1}");
            Console.ReadKey();
        }
    }
}
```

OUTPUT

When user enters N = 5:

```
=== Floyd's Triangle ===
Enter the number of rows: 5

Floyd's Triangle:
---------------------------------------------------
1
2       3
4       5       6
7       8       9       10
11      12      13      14      15
---------------------------------------------------
Total numbers printed: 15
```

Practical 1c: Arrange Names of Cities in Sorted Order

AIM: To write a C# program that accepts names of 10 cities from the user and displays them in sorted (alphabetical) order.

Steps To Follow:

1. Open Visual Studio → File → New → Project
2. Select 'Console App' → Name: SortCities → Click Create
3. Write the following code in Program.cs and run with Ctrl+F5

PROGRAM CODE :

```
using System;

namespace SortCities
{
   class Program
   {
      static void Main(string[] args)
      {
         string[] cities = new string[10];

         Console.WriteLine("=== City Name Sorter ===");
         Console.WriteLine("Enter names of 10 cities:");
         Console.WriteLine(new string('-', 30));

         for (int i = 0; i < 10; i++)
         {
            Console.Write($"City {i + 1}: ");
            cities[i] = Console.ReadLine();
         }

         // Sort array using Array.Sort()
         Array.Sort(cities, StringComparer.OrdinalIgnoreCase);

         Console.WriteLine("\n--- Cities in Sorted Order ---");
         for (int i = 0; i < cities.Length; i++)
         {
            Console.WriteLine($"{i + 1}. {cities[i]}");
```

```
        }

        Console.WriteLine(new string('-', 30));
        Console.ReadKey();
    }
  }
}
```

OUTPUT

```
=== City Name Sorter ===
Enter names of 10 cities:
----------------------------------------
City 1: Mumbai
City 2: Delhi
City 3: Pune
City 4: Chennai
City 5: Bangalore
City 6: Hyderabad
City 7: Kolkata
City 8: Ahmedabad
City 9: Jaipur
City 10: Surat

--- Cities in Sorted Order ---
1. Ahmedabad
2. Bangalore
3. Chennai
4. Delhi
5. Hyderabad
6. Jaipur
7. Kolkata
8. Mumbai
9. Pune
10. Surat
----------------------------------------
```

Practical 2a: Function Overloading

AIM : To demonstrate function overloading in C# by creating multiple methods with the same name but different parameters.

PROGRAM CODE

```
using System;

namespace FunctionOverloading
{
   class Calculator
   {
      // Overload 1: Two integers
      public int Add(int a, int b)
      {
         Console.WriteLine($"Add(int, int) called");
         return a + b;
      }

      // Overload 2: Three integers
      public int Add(int a, int b, int c)
      {
         Console.WriteLine($"Add(int, int, int) called");
         return a + b + c;
      }

      // Overload 3: Two doubles
      public double Add(double a, double b)
      {
         Console.WriteLine($"Add(double, double) called");
         return a + b;
      }

      // Overload 4: String concatenation
      public string Add(string a, string b)
      {
         Console.WriteLine($"Add(string, string) called");
         return a + b;
      }
```

```
    }

    class Program
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();
            Console.WriteLine("=== Function Overloading Demo ===");
            Console.WriteLine(calc.Add(10, 20));          // 30
            Console.WriteLine(calc.Add(10, 20, 30));      // 60
            Console.WriteLine(calc.Add(1.5, 2.5));        // 4.0
            Console.WriteLine(calc.Add("Hello, ", "World!"));
            Console.ReadKey();
        }
    }
}
```

Output:

```
=== Function Overloading Demo ===
Add(int, int) called
30
Add(int, int, int) called
60
Add(double, double) called
4
Add(string, string) called
Hello, World!
```

Practical 2b: Inheritance (All Types)

AIM : To demonstrate all types of inheritance in C#: Single, Multilevel, Hierarchical, and Multiple Inheritance (via Interfaces).

PROGRAM CODE

```
using System;

namespace InheritanceDemo
{
   // ===== SINGLE INHERITANCE =====
   class Animal
   {
      public void Eat() => Console.WriteLine("Animal is eating");
   }

   class Dog : Animal  // Dog inherits Animal
   {
      public void Bark() => Console.WriteLine("Dog is barking");
   }

   // ===== MULTILEVEL INHERITANCE =====
   class GrandParent
   {
      public void Method1() => Console.WriteLine("GrandParent method");
   }

   class Parent : GrandParent
   {
      public void Method2() => Console.WriteLine("Parent method");
   }

   class Child : Parent  // Child -> Parent -> GrandParent
   {
      public void Method3() => Console.WriteLine("Child method");
   }

   // ===== HIERARCHICAL INHERITANCE =====
   class Shape
```

```
{
    public void Draw() => Console.WriteLine("Drawing a shape");
}

class Circle : Shape
{
    public void Info() => Console.WriteLine("This is a Circle");
}

class Rectangle : Shape
{
    public void Info() => Console.WriteLine("This is a Rectangle");
}

// ===== MULTIPLE INHERITANCE via INTERFACES =====
interface IFlyable
{
    void Fly();
}

interface ISwimmable
{
    void Swim();
}

class Duck : IFlyable, ISwimmable
{
    public void Fly() => Console.WriteLine("Duck can fly");
    public void Swim() => Console.WriteLine("Duck can swim");
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("--- Single Inheritance ---");
        Dog d = new Dog();
        d.Eat();
        d.Bark();
```

```
        Console.WriteLine("--- Multilevel Inheritance ---");
        Child c = new Child();
        c.Method1();
        c.Method2();
        c.Method3();

        Console.WriteLine("--- Hierarchical Inheritance ---");
        Circle cir = new Circle();
        cir.Draw();
        cir.Info();

        Rectangle rec = new Rectangle();
        rec.Draw();
        rec.Info();

        Console.WriteLine("--- Multiple Inheritance (Interface) ---");
        Duck duck = new Duck();
        duck.Fly();
        duck.Swim();

        Console.ReadKey();
    }
  }
}
```

OUTPUT

```
  --- Single
Inheritance ---
Animal is eating
  Dog is barking
  --- Multilevel
Inheritance ---
GrandParent method
  Parent
method
Child
method
  --- Hierarchical
Inheritance ---Drawing a
```

Practical 2c: Constructor Overloading

AIM : To demonstrate constructor overloading in C# by defining multiple constructors with different parameter lists in the same class.

PROGRAM CODE

```
using System;

namespace ConstructorOverloading
{
   class Student
   {
      public string Name;
      public int Age;
      public string Course;

      // Default constructor
      public Student()
      {
         Name = "Unknown";
         Age = 0;
         Course = "N/A";
         Console.WriteLine("Default constructor called");
      }

      // Parameterized constructor with 1 argument
      public Student(string name)
      {
         Name = name;
         Age = 18;
         Course = "BSc IT";
         Console.WriteLine("1-param constructor called");
      }

      // Parameterized constructor with 2 arguments
      public Student(string name, int age)
      {
         Name = name;
```

```
        Age = age;
        Course = "BSc IT";
        Console.WriteLine("2-param constructor called");
    }

    // Parameterized constructor with all arguments
    public Student(string name, int age, string course)
    {
        Name = name;
        Age = age;
        Course = course;
        Console.WriteLine("3-param constructor called");
    }

    public void Display()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}, Course: {Course}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== Constructor Overloading Demo ===");
        Student s1 = new Student();
        s1.Display();

        Student s2 = new Student("Alice");
        s2.Display();

        Student s3 = new Student("Bob", 20);
        s3.Display();

        Student s4 = new Student("Carol", 21, "BCA");
        s4.Display();

        Console.ReadKey();
    }
}
```

}

OUTPUT

```
=== Constructor Overloading Demo ===
Default constructor called
Name: Unknown, Age: 0, Course: N/A
1-param constructor called
Name: Alice, Age: 18, Course: BSc IT
2-param constructor called
Name: Bob, Age: 20, Course: BSc IT
3-param constructor called
Name: Carol, Age: 21, Course: BCA
```

Practical 2d: Interfaces

AIM : To demonstrate the use of interfaces in C# — defining contracts that classes must implement.

PROGRAM CODE

```
using System;
namespace InterfaceDemo
{
   // Define interfaces
   interface IShape
   {
      double Area();
      double Perimeter();
      void Display();
   }

   interface IPrintable
   {
      void Print();
   }

   class Circle : IShape, IPrintable
   {
      private double radius;
      public Circle(double r) { radius = r; }

      public double Area() => Math.PI * radius * radius;
      public double Perimeter() => 2 * Math.PI * radius;
      public void Display()
      {
         Console.WriteLine($"Circle: Radius={radius}");
         Console.WriteLine($"   Area      = {Area():F2}");
         Console.WriteLine($"   Perimeter = {Perimeter():F2}");
      }
      public void Print() => Console.WriteLine("[Printable] Circle Info Printed.");
   }

   class Rectangle : IShape
```

```
    {
        private double length, width;
        public Rectangle(double l, double w) { length = l; width = w; }

        public double Area() => length * width;
        public double Perimeter() => 2 * (length + width);
        public void Display()
        {
            Console.WriteLine($"Rectangle: Length={length}, Width={width}");
            Console.WriteLine($"   Area      = {Area():F2}");
            Console.WriteLine($"   Perimeter = {Perimeter():F2}");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Interface Demo ===");
            IShape c = new Circle(7.0);
            c.Display();
            ((IPrintable)c).Print();

            Console.WriteLine();
            IShape r = new Rectangle(5.0, 3.0);
            r.Display();
            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Interface Demo ===
Circle: Radius=7
  Area      = 153.94
  Perimeter = 43.98
[Printable] Circle Info Printed.

Rectangle: Length=5, Width=3
  Area      = 15.00
  Perimeter = 16.00
```

Practical 2e: Using Delegates and Events

AIM: To demonstrate the use of delegates and events in C# for achieving callback mechanisms and event-driven programming.

PROGRAM CODE

```
using System;

namespace DelegatesEvents
{
   // Declare a delegate
   public delegate void NotifyHandler(string message);

   class EventPublisher
   {
      // Declare an event using the delegate
      public event NotifyHandler OnDataProcessed;

      public void ProcessData(string data)
      {
         Console.WriteLine($"Processing data: {data}");
         // Raise the event
         OnDataProcessed?.Invoke($"Data '{data}' has been processed!");
      }
   }

   class EventSubscriber
   {
      public void HandleEvent(string message)
      {
         Console.WriteLine($"[Subscriber] Received: {message}");
      }
   }

   // Simple delegate demo
   delegate int MathOperation(int x, int y);

   class Program
   {
```

```
    static int Multiply(int a, int b) => a * b;
    static int Add(int a, int b) => a + b;

    static void Main(string[] args)
    {
        Console.WriteLine("=== Delegates Demo ===");
        MathOperation op = Multiply;
        Console.WriteLine($"Multiply(4,5) = {op(4, 5)}");

        op = Add;
        Console.WriteLine($"Add(4,5) = {op(4, 5)}");

        Console.WriteLine("\n=== Events Demo ===");
        EventPublisher publisher = new EventPublisher();
        EventSubscriber subscriber = new EventSubscriber();

        // Subscribe to the event
        publisher.OnDataProcessed += subscriber.HandleEvent;
        publisher.OnDataProcessed += (msg) => Console.WriteLine($"[Lambda] {msg}");

        publisher.ProcessData("Student Record");
        publisher.ProcessData("Salary Data");

        Console.ReadKey();
    }
  }
}
```

OUTPUT

```
=== Delegates Demo ===
Multiply(4,5) = 20
Add(4,5) = 9

=== Events Demo ===
Processing data: Student Record
[Subscriber] Received: Data 'Student Record' has been processed!
[Lambda] Data 'Student Record' has been processed!
Processing data: Salary Data
[Subscriber] Received: Data 'Salary Data' has been processed!
[Lambda] Data 'Salary Data' has been processed!
```

Practical 2f: Exception Handling

AIM: To demonstrate exception handling in C# using try-catch-finally blocks, custom exceptions, and multiple catch handlers.

PROGRAM CODE

```
using System;

namespace ExceptionHandling
{
   // Custom exception
   class InvalidAgeException : Exception
   {
      public InvalidAgeException(string msg) : base(msg) { }
   }

   class Program
   {
      static void ValidateAge(int age)
      {
         if (age < 0 || age > 120)
            throw new InvalidAgeException($"Invalid age: {age}");
      }

      static void Main(string[] args)
      {
         // 1. DivideByZeroException
         Console.WriteLine("--- DivideByZero Exception ---");
         try
         {
            int a = 10, b = 0;
            Console.WriteLine(a / b);
         }
         catch (DivideByZeroException ex)
         {
            Console.WriteLine($"Caught: {ex.Message}");
         }
         finally
         {
```

```
            Console.WriteLine("Finally block executed.");
        }

        // 2. FormatException
        Console.WriteLine("\n--- Format Exception ---");
        try
        {
            int num = int.Parse("abc");
        }
        catch (FormatException ex)
        {
            Console.WriteLine($"Caught: {ex.Message}");
        }

        // 3. IndexOutOfRange
        Console.WriteLine("\n--- IndexOutOfRange Exception ---");
        try
        {
            int[] arr = { 1, 2, 3 };
            Console.WriteLine(arr[10]);
        }
        catch (IndexOutOfRangeException ex)
        {
            Console.WriteLine($"Caught: {ex.Message}");
        }

        // 4. Custom Exception
        Console.WriteLine("\n--- Custom Exception ---");
        try
        {
            ValidateAge(-5);
        }
        catch (InvalidAgeException ex)
        {
            Console.WriteLine($"Custom Exception: {ex.Message}");
        }

        Console.ReadKey();
    }
}
```

}

OUTPUT

```
--- DivideByZero Exception ---
Caught: Attempted to divide by zero.
Finally block executed.

--- Format Exception ---
Caught: Input string was not in a correct format.

--- IndexOutOfRange Exception ---
Caught: Index was outside the bounds of the array.

--- Custom Exception ---
Custom Exception: Invalid age: -5
```

Practical 3a: Arithmetic Methods via C# Delegates

AIM : To create arithmetic methods add(), multiply(), subtract(), divide() and call them using C# delegates.

PROGRAM CODE

```
using System;

namespace DelegateArithmetic
{
   delegate double MathOp(double a, double b);

   class Arithmetic
   {
      public static double Add(double a, double b)      => a + b;
      public static double Multiply(double a, double b) => a * b;
      public static double Subtract(double a, double b) => a - b;
      public static double Divide(double a, double b)
      {
         if (b == 0) throw new DivideByZeroException();
         return a / b;
      }
   }

   class Program
   {
      static void Main(string[] args)
      {
         Console.WriteLine("=== Arithmetic via Delegates ===");
         double x = 10, y = 5;

         MathOp op = Arithmetic.Add;
         Console.WriteLine($"Add({x},{y})       = {op(x, y)}");

         op = Arithmetic.Subtract;
         Console.WriteLine($"Subtract({x},{y}) = {op(x, y)}");

         op = Arithmetic.Multiply;
         Console.WriteLine($"Multiply({x},{y}) = {op(x, y)}");
```

```
            op = Arithmetic.Divide;
            Console.WriteLine($"Divide({x},{y})   = {op(x, y)}");

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Arithmetic via Delegates ===
Add(10,5)      = 15
Subtract(10,5) = 5
Multiply(10,5) = 50
Divide(10,5)   = 2
```

Practical 3b: Multicast Delegate

AIM : To demonstrate a multicast delegate in C# that holds references to multiple methods and invokes them all in sequence.

PROGRAM CODE

```
using System;

namespace MulticastDelegate
{
   delegate void PrintDelegate(string message);

   class Printer
   {
      public static void PrintUpperCase(string msg)
      {
         Console.WriteLine($"UPPER: {msg.ToUpper()}");
      }

      public static void PrintLowerCase(string msg)
      {
         Console.WriteLine($"lower: {msg.ToLower()}");
      }

      public static void PrintReverse(string msg)
      {
         char[] chars = msg.ToCharArray();
         Array.Reverse(chars);
         Console.WriteLine($"Reversed: {new string(chars)}");
      }
   }

   class Program
   {
      static void Main(string[] args)
      {
         Console.WriteLine("=== Multicast Delegate Demo ===");

         PrintDelegate del = Printer.PrintUpperCase;
```

```
        del += Printer.PrintLowerCase;
        del += Printer.PrintReverse;

        Console.WriteLine("Invoking multicast delegate:");
        del("Hello World");

        Console.WriteLine("\nRemoving PrintLowerCase:");
        del -= Printer.PrintLowerCase;
        del("Hello World");

        Console.ReadKey();
    }
  }
}
```

OUTPUT

```
=== Multicast Delegate Demo ===
Invoking multicast delegate:
UPPER: HELLO WORLD
lower: hello world
Reversed: dlroW olleH
```

```
Removing PrintLowerCase:
UPPER: HELLO WORLD
Reversed: dlroW olleH
```

Practical 3c: BankAccount Class with Property Validation

AIM : To create a BankAccount class with AccountNumber and Balance properties, where the Balance property includes validation to prevent negative balances.

PROGRAM CODE

```
using System;

namespace BankAccountApp
{
   class BankAccount
   {
      private string accountNumber;
      private double balance;

      public string AccountNumber
      {
         get { return accountNumber; }
         set
         {
            if (string.IsNullOrEmpty(value))
               throw new ArgumentException("Account number cannot be empty!");
            accountNumber = value;
         }
      }

      public double Balance
      {
         get { return balance; }
         set
         {
            if (value < 0)
               throw new ArgumentException("Balance cannot be negative!");
            balance = value;
         }
      }

      public BankAccount(string accNo, double initialBalance)
      {
```

```
        AccountNumber = accNo;
        Balance = initialBalance;
    }

    public void Deposit(double amount)
    {
        if (amount <= 0) throw new ArgumentException("Deposit must be positive.");
        Balance += amount;
        Console.WriteLine($"Deposited Rs.{amount}. New Balance: Rs.{Balance}");
    }

    public void Withdraw(double amount)
    {
        if (amount > Balance) throw new InvalidOperationException("Insufficient funds!");

        Balance -= amount;
        Console.WriteLine($"Withdrawn Rs.{amount}. New Balance: Rs.{Balance}");
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Account: {AccountNumber} | Balance: Rs.{Balance}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== Bank Account Demo ===");
        BankAccount acc = new BankAccount("ACC1001", 5000);
        acc.DisplayInfo();
        acc.Deposit(2000);
        acc.Withdraw(1500);
        acc.DisplayInfo();

        Console.WriteLine("\nTrying negative balance:");
        try { acc.Balance = -100; }
        catch (Exception ex) { Console.WriteLine($"Error: {ex.Message}"); }
```

```
        Console.WriteLine("\nTrying overdraft:");
        try { acc.Withdraw(50000); }
        catch (Exception ex) { Console.WriteLine($"Error: {ex.Message}"); }

        Console.ReadKey();
    }
  }
}
```

OUTPUT

```
=== Bank Account Demo ===
Account: ACC1001 | Balance: Rs.5000
Deposited Rs.2000. New Balance: Rs.7000
Withdrawn Rs.1500. New Balance: Rs.5500
Account: ACC1001 | Balance: Rs.5500

Trying negative balance:
Error: Balance cannot be negative!

Trying overdraft:
Error: Insufficient funds!
```

Practical 4a: Student Registration Form

AIM : To design a Student Registration Form using ASP.NET Web Forms controls including Label, TextBox, RadioButton, DropDownList, CheckBox, Calendar, and Button.

Steps To Follow:

1. Open Visual Studio → File → New → Project → ASP.NET Web Application (.NET Framework)
2. Select 'Web Forms' template → Name: StudentRegForm → Click Create
3. Open Default.aspx and add the following markup
4. Open Default.aspx.cs and add the code-behind logic
5. Press F5 to run the application

PROGRAM CODE

Default.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
   CodeBehind="Default.aspx.cs" Inherits="StudentRegForm.Default" %>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Student Registration</title>
   <style>
      body { font-family: Arial; margin: 30px; }
      .form-group { margin: 10px 0; }
      label { display: inline-block; width: 180px; font-weight: bold; }
   </style>
</head>
<body>
   <form id="form1" runat="server">
      <h2>Student Registration Form</h2>
      <hr />
      <div class="form-group">
         <asp:Label Text="Student Name:" runat="server" />
         <asp:TextBox ID="txtName" runat="server" Width="200px" />
      </div>
      <div class="form-group">
         <asp:Label Text="Gender:" runat="server" />
```

```
        <asp:RadioButton ID="rbMale" GroupName="gender" Text="Male" runat="server" />
        <asp:RadioButton ID="rbFemale" GroupName="gender" Text="Female" runat="server"
/>
      </div>
      <div class="form-group">
        <asp:Label Text="Course:" runat="server" />
        <asp:DropDownList ID="ddlCourse" runat="server">
          <asp:ListItem Text="Select Course" Value="" />
          <asp:ListItem Text="BSc IT" Value="BSc IT" />
          <asp:ListItem Text="BSc CS" Value="BSc CS" />
          <asp:ListItem Text="BCA" Value="BCA" />
        </asp:DropDownList>
      </div>
      <div class="form-group">
        <asp:Label Text="Date of Birth:" runat="server" />
        <asp:Calendar ID="calDOB" runat="server" />
      </div>
      <div class="form-group">
        <asp:CheckBox ID="chkTerms" Text="I accept Terms and Conditions" runat="server"
/>
      </div>
      <div class="form-group">
        <asp:Button ID="btnSubmit" Text="Register" runat="server"
          OnClick="btnSubmit_Click" />
        <asp:Button ID="btnClear" Text="Clear" runat="server"
          OnClick="btnClear_Click" />
      </div>
      <hr />
      <asp:Label ID="lblOutput" runat="server" ForeColor="Green" Font-Bold="true" />
    </form>
</body>
</html>
```

Default.aspx.cs

```
using System;

namespace StudentRegForm
{
    public partial class Default : System.Web.UI.Page
```

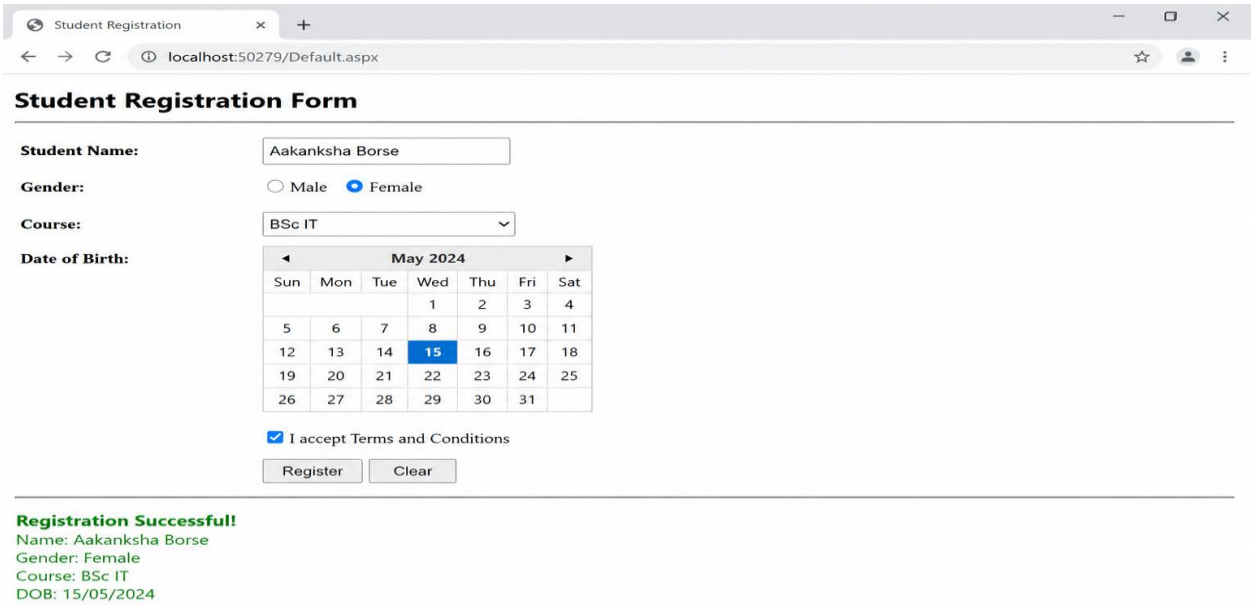
```
    {
        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            if (!chkTerms.Checked)
            {
                lblOutput.ForeColor = System.Drawing.Color.Red;
                lblOutput.Text = "Please accept Terms and Conditions!";
                return;
            }

            string gender = rbMale.Checked ? "Male" : (rbFemale.Checked ? "Female" : "Not
Selected");

            string dob = calDOB.SelectedDate == DateTime.MinValue
                ? "Not Selected"
                : calDOB.SelectedDate.ToShortDateString();
            lblOutput.ForeColor = System.Drawing.Color.Green;
            lblOutput.Text = $"Registration Successful!<br/>"
                + $"Name: {txtName.Text}<br/>"
                + $"Gender: {gender}<br/>"
                + $"Course: {ddlCourse.SelectedValue}<br/>"
                + $"DOB: {dob}";
        }

        protected void btnClear_Click(object sender, EventArgs e)
        {
            txtName.Text = "";
            rbMale.Checked = false;
            rbFemale.Checked = false;
            ddlCourse.SelectedIndex = 0;
            chkTerms.Checked = false;
            lblOutput.Text = "";
        }
    }
}
```

Output:-


Student Registration
localhost:50279/Default.aspx
Student Registration Form
Student Name:
Aakanksha Borse
Gender:
Male
Female
Course:
BSc IT
Date of Birth:
May 2024
Sun Mon Tue Wed Thu Fri Sat
1 2 3 4
5 6 7 8 9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
I accept Terms and Conditions
Register
Clear
Registration Successful!
Name: Aakanksha Borse
Gender: Female
Course: BSc IT
DOB: 15/05/2024

Practical 4b: Registration Form with Validation Controls

AIM : To create a Registration Form using ASP.NET validation controls: RequiredFieldValidator, RangeValidator, RegularExpressionValidator, and CompareValidator.

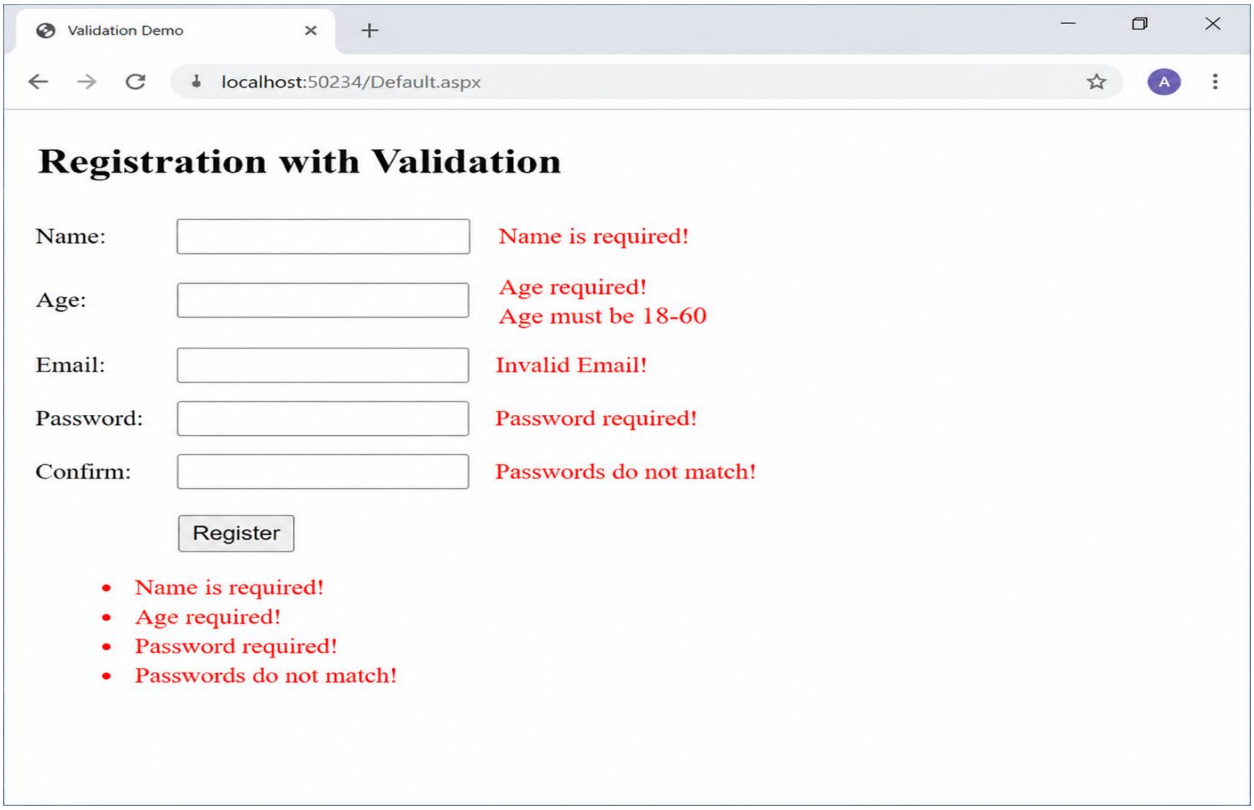
PROGRAM CODE

ValidationForm.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Validation.aspx.cs"
   Inherits="StudentRegForm.Validation" %>
<html>
<head runat="server"><title>Validation Demo</title></head>
<body>
   <form id="form1" runat="server">
      <h2>Registration with Validation</h2>
      <table>
         <tr>
            <td>Name:</td>
            <td><asp:TextBox ID="txtName" runat="server" /></td>
            <td>
               <asp:RequiredFieldValidator ControlToValidate="txtName"
                  ErrorMessage="Name is required!" ForeColor="Red" runat="server" />
            </td>
         </tr>
         <tr>
            <td>Age:</td>
            <td><asp:TextBox ID="txtAge" runat="server" /></td>
            <td>
               <asp:RequiredFieldValidator ControlToValidate="txtAge"
                  ErrorMessage="Age required!" ForeColor="Red" runat="server" />
               <asp:RangeValidator ControlToValidate="txtAge" Type="Integer"
                  MinimumValue="18" MaximumValue="60"
                  ErrorMessage="Age must be 18-60" ForeColor="Red" runat="server" />
            </td>
         </tr>
         <tr>
            <td>Email:</td>
            <td><asp:TextBox ID="txtEmail" runat="server" /></td>
            <td>
```

```
            <asp:RegularExpressionValidator ControlToValidate="txtEmail"
              ValidationExpression="^[\w-\.]+\@([\w-]+\.)+[\w-]{2,4}$"
              ErrorMessage="Invalid Email!" ForeColor="Red" runat="server" />
          </td>
        </tr>
        <tr>
          <td>Password:</td>
          <td><asp:TextBox ID="txtPass" TextMode="Password" runat="server" /></td>
          <td>
            <asp:RequiredFieldValidator ControlToValidate="txtPass"
              ErrorMessage="Password required!" ForeColor="Red" runat="server" />
          </td>
        </tr>
        <tr>
          <td>Confirm:</td>
          <td><asp:TextBox ID="txtConfirm" TextMode="Password" runat="server" /></td>
          <td>
            <asp:CompareValidator ControlToValidate="txtConfirm"
              ControlToCompare="txtPass"
              ErrorMessage="Passwords do not match!" ForeColor="Red" runat="server" />
          </td>
        </tr>
        <tr>
          <td colspan="3">
            <asp:Button ID="btnReg" Text="Register" runat="server"
              OnClick="btnReg_Click" />
          </td>
        </tr>
      </table>
      <asp:ValidationSummary runat="server" ForeColor="Red" />
      <asp:Label ID="lblMsg" runat="server" ForeColor="Green" />
    </form>
</body>
</html>
```

Output:


Validation Demo
localhost:50234/Default.aspx
Registration with Validation
Name:
Name is required!
Age:
Age required!
Age must be 18-60
Email:
Invalid Email!
Password:
Password required!
Confirm:
Passwords do not match!
Register
Name is required!
Age required!
Password required!
Passwords do not match!

Practical 5a: Different Types of Cookies

AIM: To create an ASP.NET Core web application demonstrating Session Cookies and Persistent Cookies.

CookiesController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace CookieDemo.Controllers
{
 public class CookieController : Controller
 {
    public IActionResult Index()
    {
       return View();
    }

    public IActionResult CreateCookie()
    {
       CookieOptions option = new CookieOptions();
       option.Expires = DateTime.Now.AddDays(7);

       Response.Cookies.Append("Username", "Aditya", option);

       return Content("Cookie Created Successfully");
    }

    public IActionResult ReadCookie()
    {
       string? value = Request.Cookies["Username"];

       if (string.IsNullOrEmpty(value))
          return Content("Cookie Not Found");

       return Content("Cookie Value: " + value);
    }

    public IActionResult DeleteCookie()
    {
```

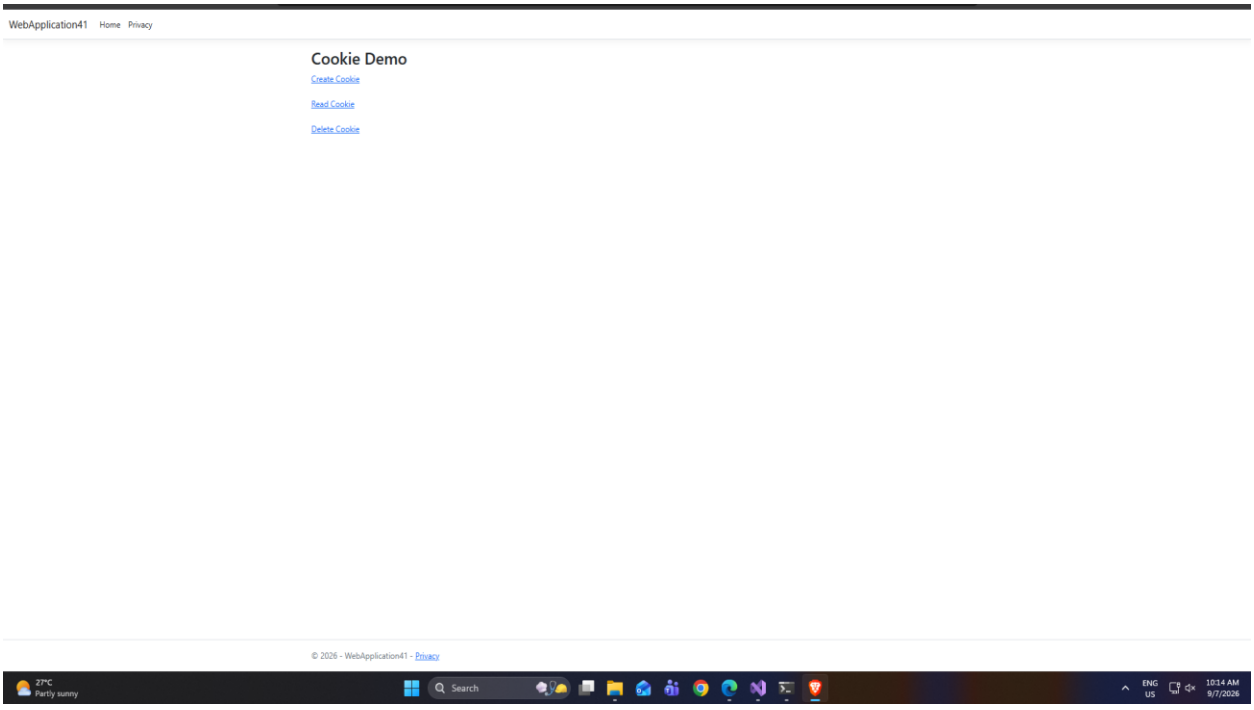
```
            Response.Cookies.Delete("Username");
            return Content("Cookie Deleted");
        }
    }
}
```

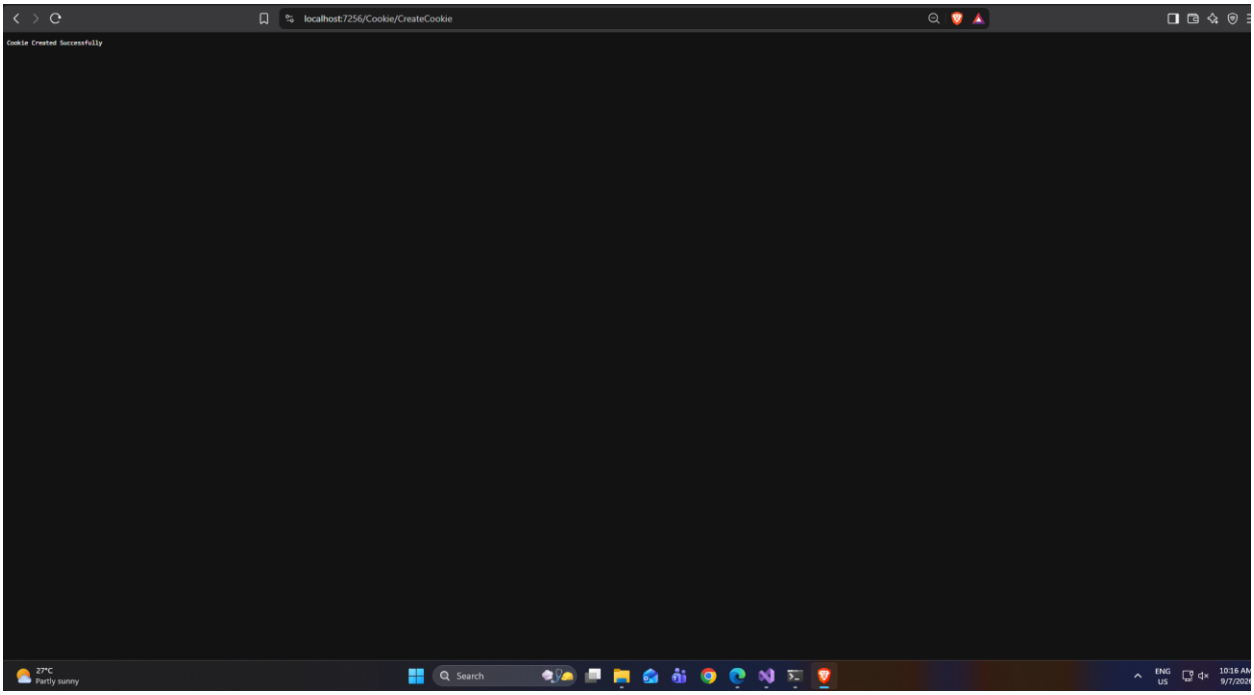
Index.cshtml

```
<h2>Cookie Demo</h2>

<a href="/Cookie/CreateCookie">Create Cookie</a>
<br />
<br />
<a href="/Cookie/ReadCookie">Read Cookie</a>
<br />
<br />
<a href="/Cookie/DeleteCookie">Delete Cookie</a>
```

Output




localhost:7256/Cookie/CreateCookie
Cookie Created Successfully
27°C
Partly sunny
Search
ENG
US

Practical 5b: Product Table with EF Core (Insert Operation)

AIM: To create a Product table in SQL Server and perform Insert Operation using Entity Framework Core in an ASP.NET Core application.

Steps

1. Install NuGet packages: Microsoft.EntityFrameworkCore, Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools
2. Create the Product model and DbContext
3. Configure connection string in appsettings.json
4. Run migrations: Add-Migration Init → Update-Database

Product.cs

```
using System.ComponentModel.DataAnnotations;
namespace WebApplication5.Models
{
   public class Product
   {
      [Key]
      public int Id { get; set; }

      [Required]
      public string Name { get; set; } = string.Empty;

      public decimal Price { get; set; }

      public int Quantity { get; set; }
   }
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using WebApplication5.Models;
namespace WebApplication5.Models.Data
{
   public class AppDbContext : DbContext
   {
      public AppDbContext(DbContextOptions<AppDbContext> options)
         : base(options)
      {
      }
```

```
        public DbSet<Product> Products { get; set; }
    }
}
```

ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using WebApplication5.Models;
using WebApplication5.Models.Data;

namespace WebApplication5.Controllers
{
    public class ProductController : Controller
    {
        private readonly AppDbContext _context;

        public ProductController(AppDbContext context)
        {
            _context = context;
        }

        // Display Form
        public IActionResult Index()
        {
            return View();
        }

        // Insert Product
        [HttpPost]
        public IActionResult Index(Product product)
        {
            if (ModelState.IsValid)
            {
                _context.Products.Add(product);
                _context.SaveChanges();

                ViewBag.Message = "Product Inserted Successfully";
                ModelState.Clear();
            }

            return View();
        }
    }
}
```

Appsetting Json file

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;Multiple
ActiveResultSets=true;TrustServerCertificate=True"
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using WebApplication5.Models.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Index.cshtml

```
@model WebApplication5.Models.Product
@{
    ViewData["Title"] = "Product";
}
<h2>Insert Product</h2>

<form asp-action="Index" method="post">
```

```
    <div>
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
    </div>

    <br />

    <div>
        <label asp-for="Price"></label>
        <input asp-for="Price" class="form-control" />
    </div>

    <br />

    <div>
        <label asp-for="Quantity"></label>
        <input asp-for="Quantity" class="form-control" />
    </div>

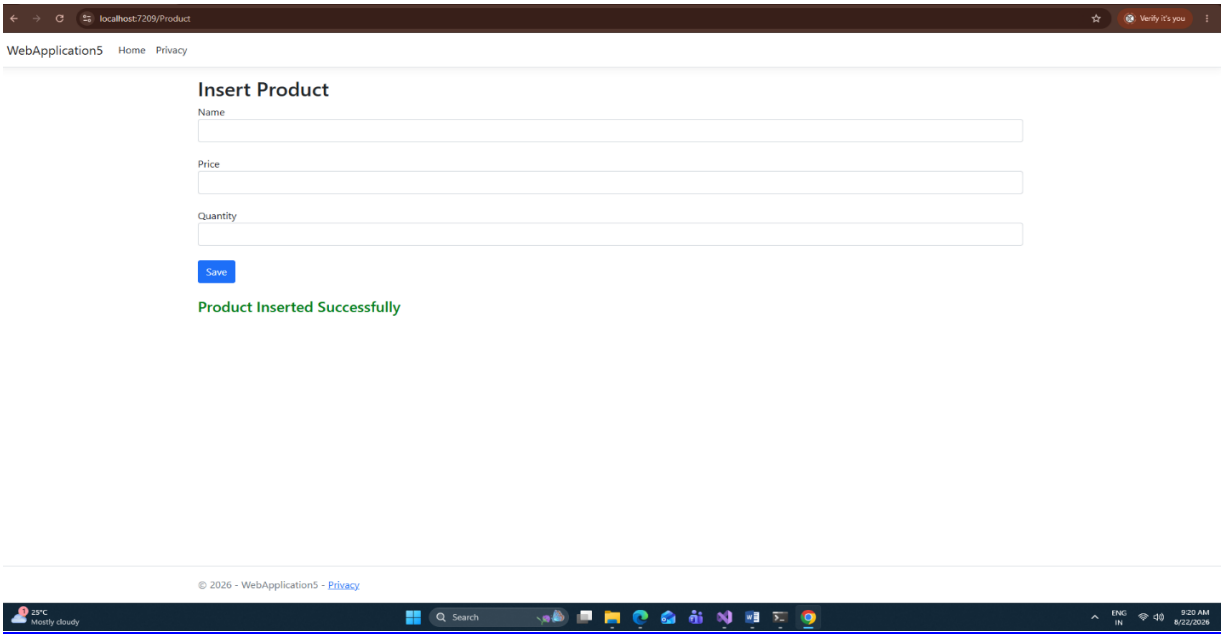
    <br />

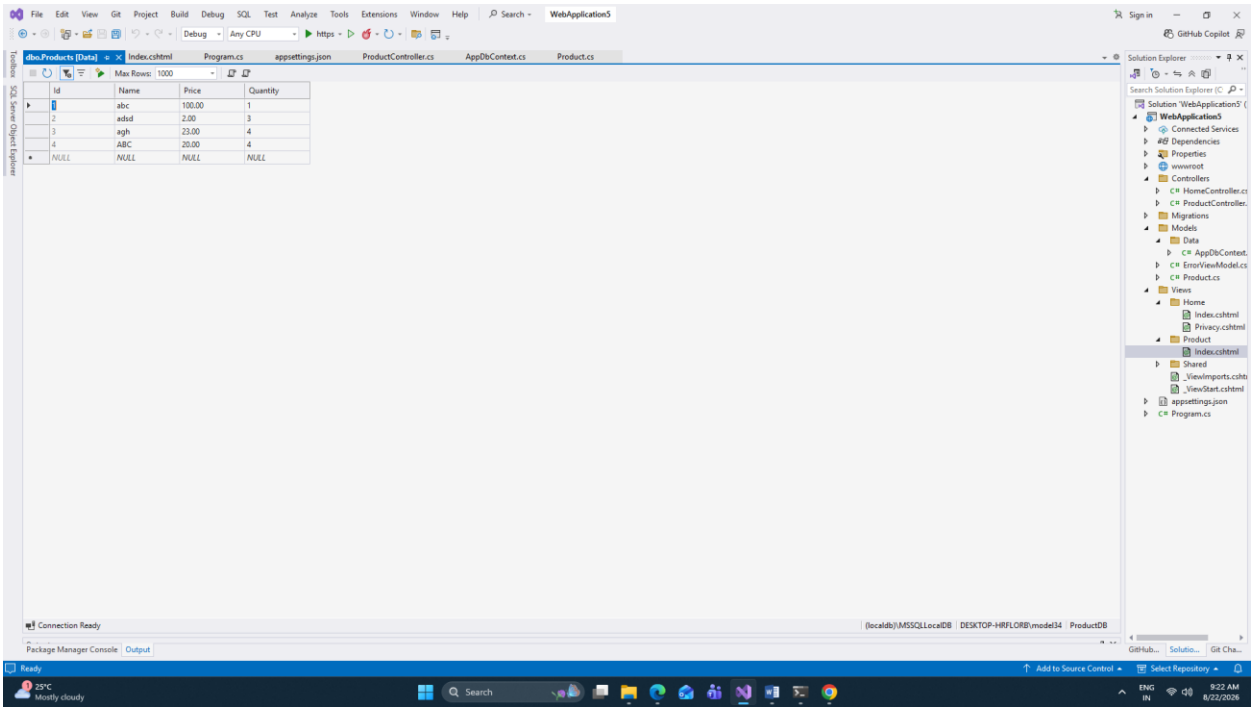
    <button type="submit" class="btn btn-primary">
        Save
    </button>

</form>

@if (ViewBag.Message != null)
{
    <br />
    <h4 style="color:green">@ViewBag.Message</h4>
}
```

Output


localhost:7209/Product
WebApplication5
Home
Privacy
Insert Product
Name
Price
Quantity
Save
Product Inserted Successfully
© 2026 - WebApplication5 - Privacy


WebApplication5
dbo.Products [Data]
Index.cshtml
Program.cs
appsettings.json
ProductController.cs
AppDbContext.cs
Product.cs
Id
Name
Price
Quantity
abc
adsd
agh
ABC
Connection Ready
Package Manager Console
Output
Ready

Module -2

Practical 1a: College Information Portal as PWA

AIM: To create a simple website for a College Information Portal and convert it into a Progressive Web Application (PWA) by adding a Service Worker and Web App Manifest.

Steps :

1. Create an HTML/CSS website for the College Portal
2. Create manifest.json with app metadata
3. Create service-worker.js to cache assets
4. Register the service worker in index.html

Index.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">

   <meta name="viewport"
       content="width=device-width, initial-scale=1.0">

   <title>College Information Portal</title>

   <!-- PWA Manifest -->
   <link rel="manifest" href="manifest.json">

   <!-- CSS -->
   <link rel="stylesheet" href="style.css">
</head>

<body>

   <header>
      <h1>College Information Portal</h1>
      <p>Welcome to Our College</p>
   </header>

   <nav>
      <a href="#home">Home</a>
```

```
    <a href="#courses">Courses</a>
    <a href="#notice">Notice</a>
    <a href="#contact">Contact</a>
</nav>

<main>

    <section id="home">
        <h2>About College</h2>
        <p>
            Welcome to our College Information Portal.
            This website provides college information,
            courses, notices and contact details.
        </p>
    </section>

    <section id="courses">
        <h2>Courses Offered</h2>

        <ul>
            <li>B.Sc. Computer Science</li>
            <li>B.Sc. Information Technology</li>
            <li>B.Com.</li>
            <li>B.A.</li>
        </ul>
    </section>

    <section id="notice">
        <h2>Latest Notices</h2>

        <p>□ Admission process has started.</p>
        <p>□ Internal examination timetable released.</p>
        <p>□ College annual event will be conducted soon.</p>
    </section>

    <section id="contact">
        <h2>Contact Us</h2>

        <p>Email: college@example.com</p>
        <p>Phone: 9876543210</p>
```

```
        </section>

    </main>

    <footer>
        <p>© 2026 College Information Portal</p>
    </footer>


    <!-- Register Service Worker -->
    <script>
        if ('serviceWorker' in navigator) {
            window.addEventListener('load', function () {
                navigator.serviceWorker.register('/service-worker.js')
                    .then(function (registration) {
                        console.log('Service Worker registered:', registration.scope);
                    })
                    .catch(function (error) {
                        console.log('Service Worker registration failed:', error);
                    });
            });
        }
    </script>

</body>
</html>
```

Style.css

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
}

header {
    background-color: #0066cc;
    color: white;
    text-align: center;
    padding: 25px;
}
```

```
nav {
    background-color: #333;
    text-align: center;
    padding: 15px;
}

    nav a {
        color: white;
        text-decoration: none;
        margin: 15px;
    }

main {
    width: 80%;
    margin: 20px auto;
}

section {
    background-color: white;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
}

h2 {
    color: #0066cc;
}

footer {
    background-color: #333;
    color: white;
    text-align: center;
    padding: 15px;
}
```

Service-Worker.js

```
const CACHE_NAME = "college-portal-v1";

const ASSETS = [
    "/",
    "/css/site.css",
    "/manifest.json"
];
```

```
self.addEventListener("install", event => {
   console.log("Service Worker: Installing...");

   event.waitUntil(
      caches.open(CACHE_NAME)
         .then(cache => {
            return cache.addAll(ASSETS);
         })
   );
});

self.addEventListener("activate", event => {
   console.log("Service Worker: Activated");
});

self.addEventListener("fetch", event => {
   event.respondWith(
      caches.match(event.request)
         .then(response => {
            return response || fetch(event.request);
         })
   );
});
```

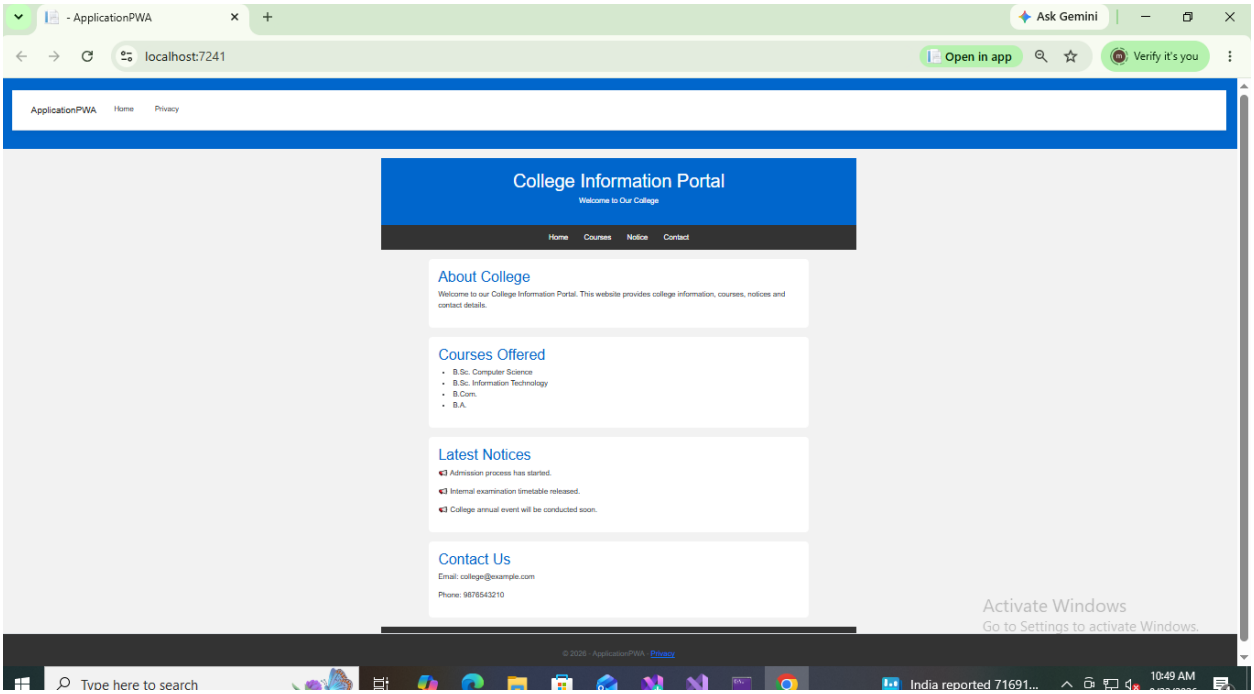
Manifest.json

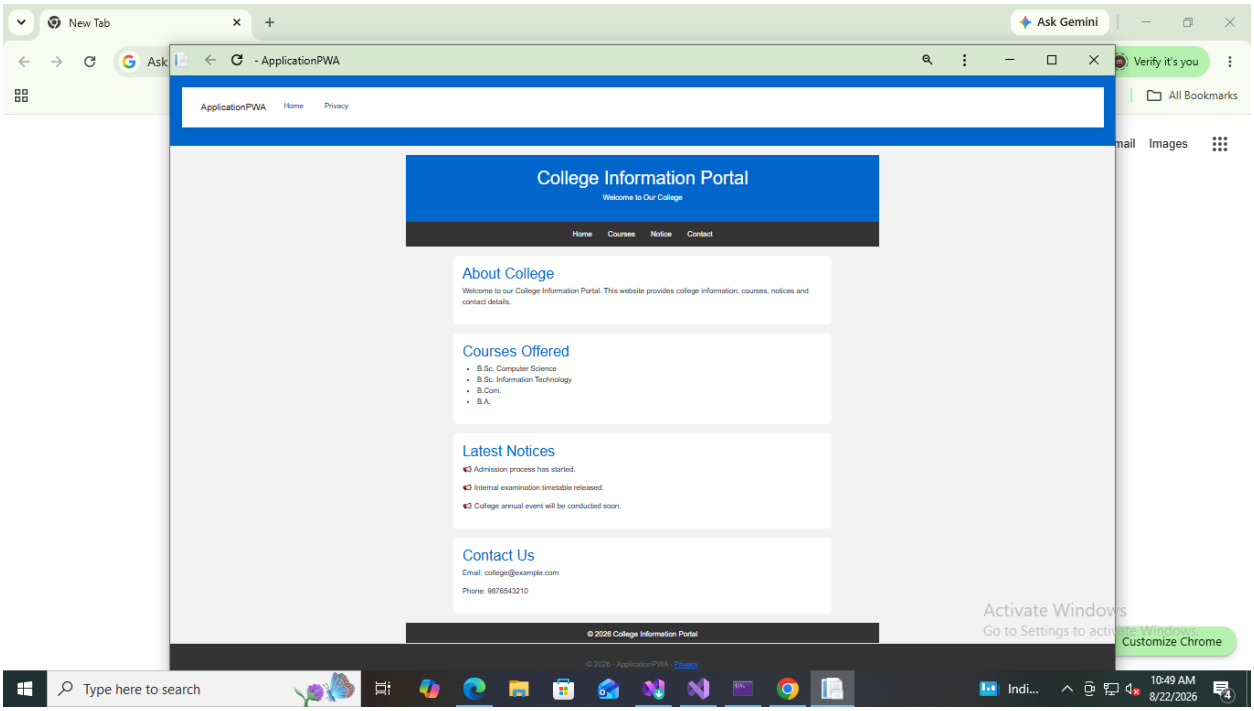
```
{
  "name": "College Information Portal",
  "short_name": "College Portal",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0066cc",
  "description": "College Information Portal Progressive Web Application",

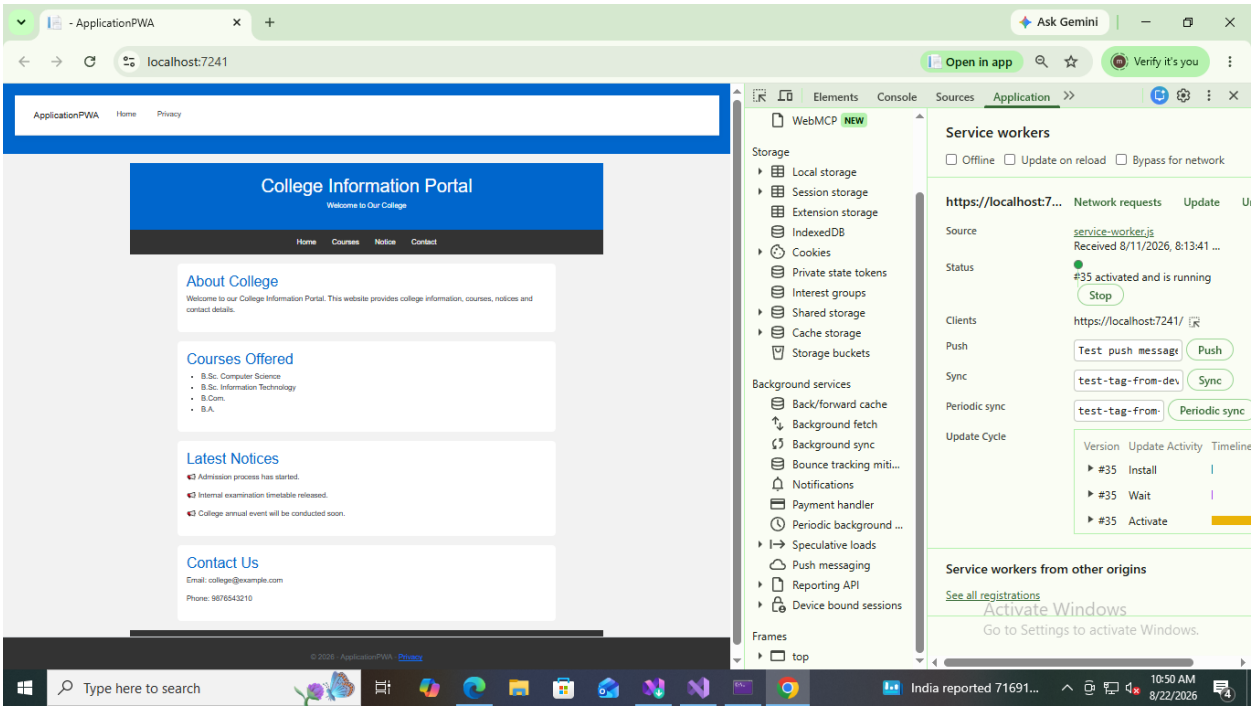
  "icons": [
    {
      "src": "icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon-512.png",
      "sizes": "512x512",
```

```
    "type": "image/png"
   }
  ]
}
```

Output




New Tab
Ask Gemini
- ApplicationPWA
Verify it's you
All Bookmarks
ApplicationPWA
Home
Privacy
College Information Portal
Welcome to Our College
Home
Courses
Notice
Contact
About College
Welcome to our College Information Portal. This website provides college information, courses, notices and contact details.
Courses Offered
B.Sc. Computer Science
B.Sc. Information Technology
B.Com.
B.A.
Latest Notices
Admission process has started.
Internal examination timetable released.
College annual event will be conducted soon.
Contact Us
Email: college@example.com
Phone: 9876543210
© 2026 College Information Portal
Activate Windows
Customize Chrome
Type here to search
10:49 AM
8/22/2026


- ApplicationPWA
Ask Gemini
localhost:7241
Open in app
Verify it's you
ApplicationPWA
Home
Privacy
College Information Portal
Welcome to Our College
Home
Courses
Notice
Contact
About College
Courses Offered
Latest Notices
Contact Us
Elements
Console
Sources
Application
WebMCP
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
Private state tokens
Interest groups
Shared storage
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Bounce tracking miti...
Notifications
Payment handler
Periodic background ...
Speculative loads
Push messaging
Reporting API
Device bound sessions
Frames
top
Service workers
Offline
Update on reload
Bypass for network
https://localhost:7...
Network requests
Update
Source
service-worker.js
Received 8/11/2026, 8:13:41 ...
Status
#35 activated and is running
Stop
Clients
https://localhost:7241/
Push
Test push message
Push
Sync
test-tag-from-dev
Sync
Periodic sync
test-tag-from-
Periodic sync
Update Cycle
Version
Update Activity
Timeline
#35 Install
#35 Wait
#35 Activate
Service workers from other origins
See all registrations
Activate Windows
Go to Settings to activate Windows.
Type here to search
India reported 71691...
10:50 AM
8/22/2026

Practical1b: PWA with Custom Offline Page

AIM : To create a PWA that displays a custom offline page when the user is not connected to the internet.

Offline.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Offline - ABC College</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      padding: 60px;
      background: #f0f0f0;
    }
    .icon {
      font-size: 80px;
    }
    h1 {
      color: #1F4E79;
    }
    p {
      color: #555;
    }
    button {
      background: #1F4E79;
      color: white;
      border: none;
      padding: 12px 24px;
      cursor: pointer;
      border-radius: 4px;
    }
  </style>
</head>
<body>
  <div class="icon">□</div>
  <h1>You are Offline</h1>
```

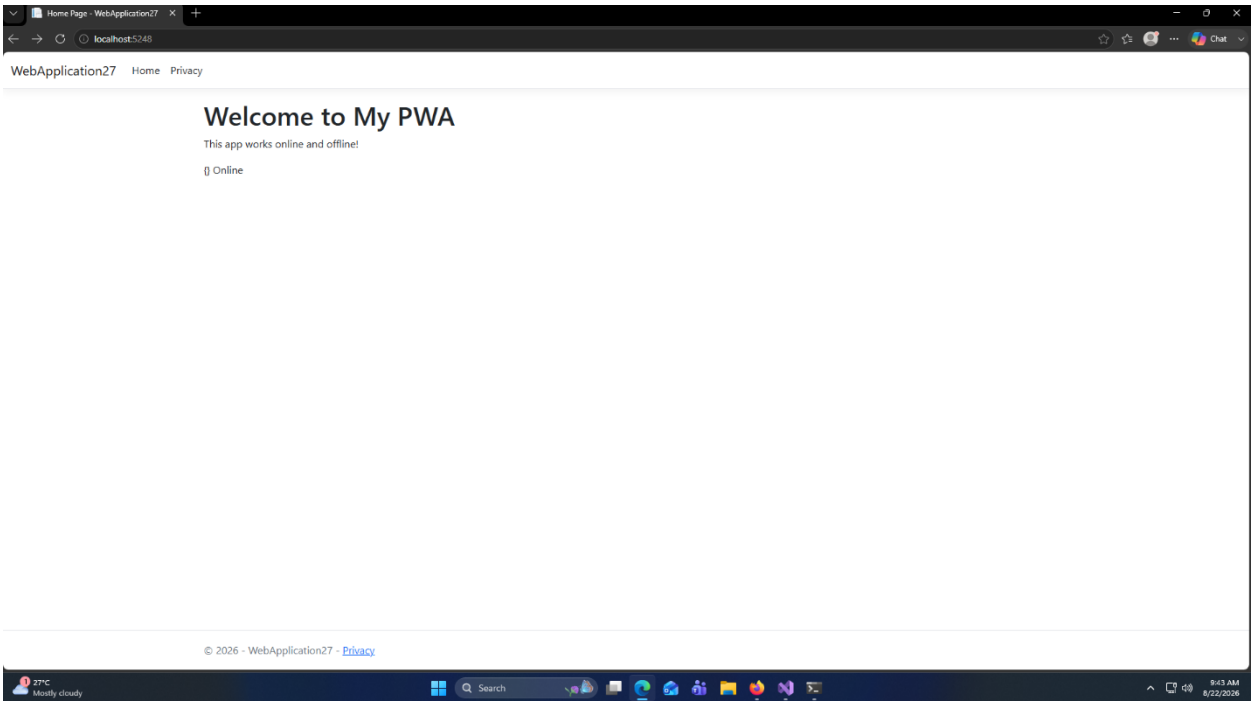
```
    <p>No internet connection found. Please check your network settings.</p>
    <p>Some content may still be available from cache.</p>
    <button onclick="window.location.reload()">Try Again</button>
</body>
</html>
```

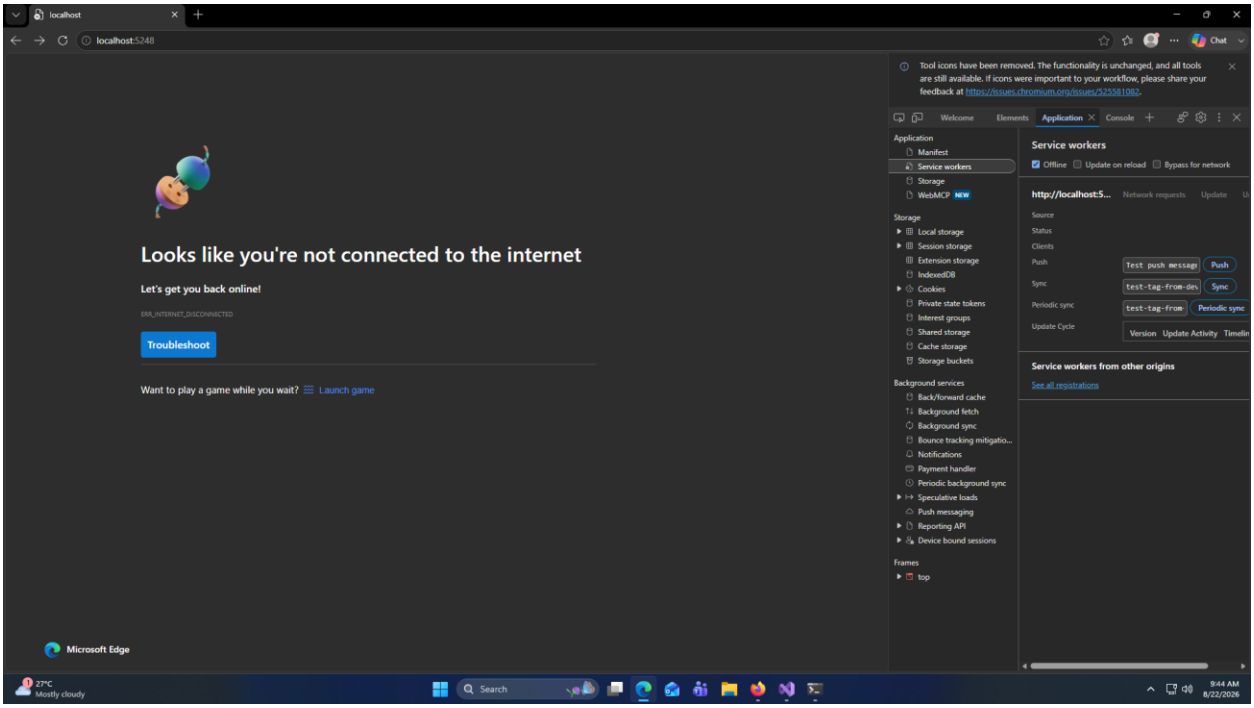
service-worker.js (with offline support)

```
const CACHE_NAME = 'pwa-offline-v1';
const OFFLINE_URL = '/offline.html';

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(['/', '/offline.html', '/style.css']);
    })
  );
  self.skipWaiting();
});

self.addEventListener('fetch', event => {
  if (event.request.mode === 'navigate') {
    event.respondWith(
      fetch(event.request).catch(() =>
        caches.open(CACHE_NAME).then(cache => cache.match(OFFLINE_URL))
      )
    );
  } else {
    event.respondWith(
      caches.match(event.request).then(r => r || fetch(event.request))
    );
  }
});
```


Home Page - WebApplication27
localhost:5248
WebApplication27
Home
Privacy
Welcome to My PWA
This app works online and offline!
{} Online
© 2026 - WebApplication27 - Privacy


localhost
localhost:5248
Looks like you're not connected to the internet
Let's get you back online!
Troubleshoot
Want to play a game while you wait?
Launch game
Microsoft Edge
Service workers
Service workers from other origins

Practical 2a: PWA with Offline Mode using Service Worker

AIM : To develop a PWA that works in offline mode using Service Worker with cache-first strategy.

service-worker.js (Cache First Strategy)

```
const CACHE_VERSION = 'v2';
const CACHE_NAME = `pwa-cache-${CACHE_VERSION}`;

const STATIC_ASSETS = [
  '/',
  '/index.html',
  '/style.css',
  '/app.js',
  '/icons/icon-192.png',
  '/icons/icon-512.png',
  '/offline.html'
];

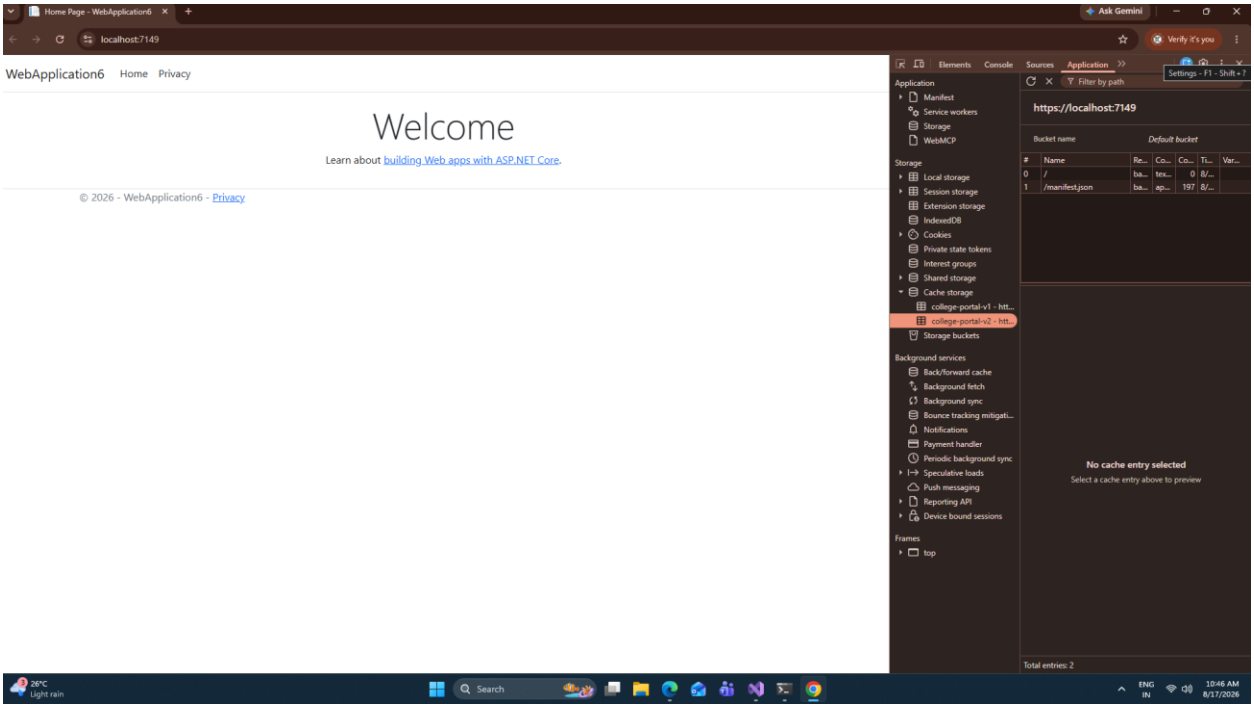
// Install: Cache all static assets
self.addEventListener('install', event => {
  console.log('[SW] Install');
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(STATIC_ASSETS))
      .then(() => self.skipWaiting())
  );
});

// Activate: Clean old caches
self.addEventListener('activate', event => {
  console.log('[SW] Activate');
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k))
      )
```

```
    ).then(() => self.clients.claim())
  );
});

// Fetch: Cache First, Network Fallback
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      if (cached) {
        console.log('[SW] Serving from cache:', event.request.url);
        return cached;
      }
      return fetch(event.request).then(response => {
        const clone = response.clone();
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, clone));
        return response;
      }).catch(() => caches.match('/offline.html'));
    })
  );
});
```

Output:



Practical 2b: Student Record Management PWA

AIM : To create a Student Record Management Progressive Web Application with offline support and

installability.

Index.html

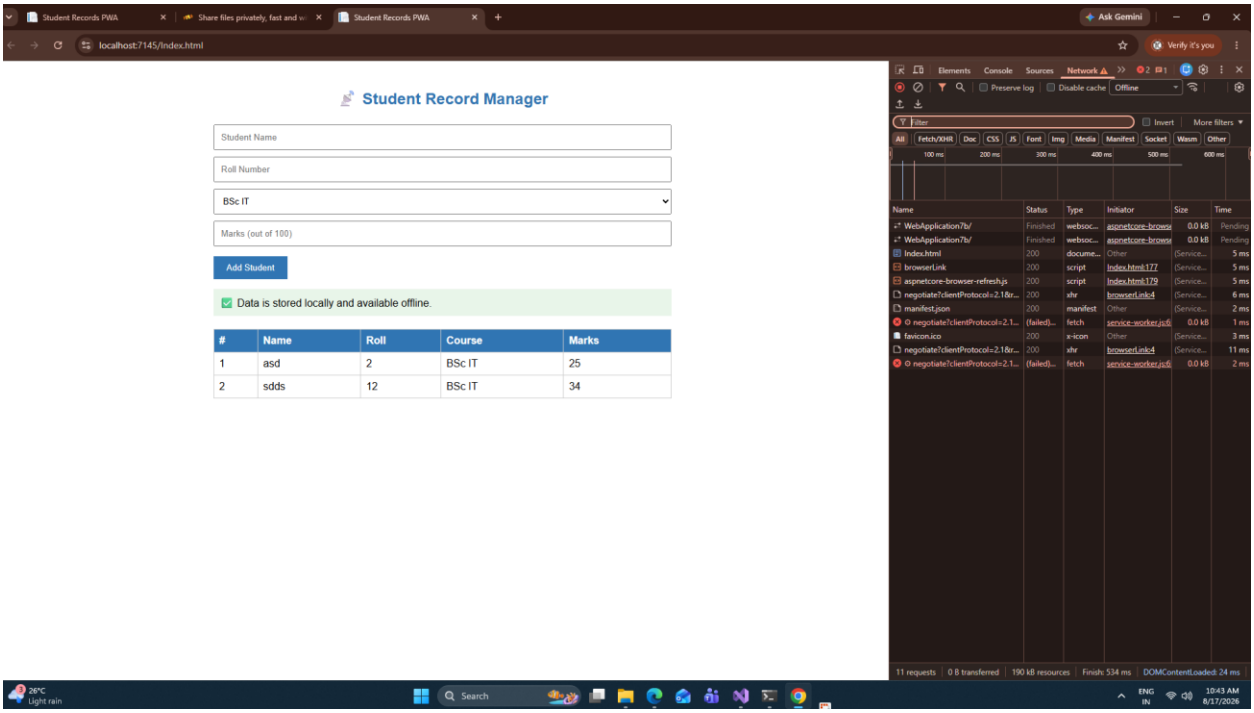
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Records PWA</title>
  <link rel="manifest" href="manifest.json" />
  <meta name="theme-color" content="#2E75B6" />
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: auto;
      padding: 20px;
    }
    input, select {
      width: 100%;
      padding: 8px;
      margin: 5px 0;
      box-sizing: border-box;
    }
    button {
      background: #2E75B6;
```

```
      color: white;
      padding: 10px 20px;
      border: none;
      cursor: pointer;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 20px;
    }
    th, td {
      border: 1px solid #ccc;
      padding: 8px;
      text-align: left;
    }
    th {
      background: #2E75B6;
      color: white;
    }
  </style>
</head>
<body>
  <h2> Student Record Manager</h2>
  <div>
    <input type="text" id="name" placeholder="Student Name" />
    <input type="text" id="roll" placeholder="Roll Number" />
    <select id="course">
      <option value="BSc IT">BSc IT</option>
      <option value="BSc CS">BSc CS</option
```

```
<option value="BCA">BCA</option>
    </select>
    <input type="number" id="marks" placeholder="Marks (out of 100)" />
    <button onclick="addStudent()">Add Student</button>
  </div>
  <table>
    <thead>
      <tr>
        <th>#</th>
        <th>Name</th>
        <th>Roll</th>
        <th>Course</th>
        <th>Marks</th>
      </tr>
    </thead>
    <tbody id="studentTable"></tbody>
  </table>
  <script>
    let students = JSON.parse(localStorage.getItem('students')) || [];
    function render() {
      const tbody = document.getElementById('studentTable');
      tbody.innerHTML = students.map((s, i) =>
        `<tr>
          <td>${i + 1}</td>
          <td>${s.name}</td>
          <td>${s.roll}</td>
          <td>${s.course}</td>
          <td>${s.marks}</td>
```

```
      </tr>`
    ).join('');
  }
  function addStudent() {
    students.push({
      name: document.getElementById('name').value,
      roll: document.getElementById('roll').value,
      course: document.getElementById('course').value,
      marks: document.getElementById('marks').value
    });
    localStorage.setItem('students', JSON.stringify(students));
    render();
  }
  render();
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/sw.js');
  }
 </script>
</body>
</html>
```

Output:


Student Records PWA
Share files privately, fast and
Student Records PWA
Ask Gemini
localhost:7145/Index.html
Verify it's you
Student Record Manager
Student Name
Roll Number
BSc IT
Marks (out of 100)
Add Student
Data is stored locally and available offline.
Name Roll Course Marks
1 asd 2 BSc IT 25
2 sdds 12 BSc IT 34
Elements Console Sources Network
Preserve log Disable cache Offline
Filter Invert More filters
All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other
100 ms 200 ms 300 ms 400 ms 500 ms 600 ms
Name Status Type Initiator Size Time
WebApplication7b/ Finished websoc... aspnetcore-browse 0.0 kB Pending
WebApplication7b/ Finished websoc... aspnetcore-browse 0.0 kB Pending
Index.html 200 docume... Other (Service... 5 ms
browserLink 200 script Index.html:177 (Service... 5 ms
aspnetcore-browser-refresh.js 200 script Index.html:179 (Service... 5 ms
negotiate?clientProtocol=2.1&r... 200 xhr browserLink:4 (Service... 6 ms
manifest.json 200 manifest Other (Service... 2 ms
negotiate?clientProtocol=2.1... (failed)... fetch service-worker.js:6 0.0 kB 1 ms
favicon.ico 200 x-icon Other (Service... 3 ms
negotiate?clientProtocol=2.1&r... 200 xhr browserLink:4 (Service... 11 ms
negotiate?clientProtocol=2.1... (failed)... fetch service-worker.js:6 0.0 kB 2 ms
11 requests 0 B transferred 190 kB resources Finish: 534 ms DOMContentLoaded: 24 ms
26°C Light rain
Search
ENG IN
10:43 AM 8/17/2026

Practical 3a: HTML Page with Link to manifest.json

AIM : To create an HTML page as the starting point of a PWA application and link it to a manifest.json file for app configuration.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="My Progressive Web App" />
  <meta name="theme-color" content="#1F4E79" />
  <!-- Link to manifest.json -->
  <link rel="manifest" href="manifest.json" />
  <!-- Icon for iOS -->
  <link rel="apple-touch-icon" href="icons/icon-192.png" />
  <title>My PWA</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div id="app">
    <h1>Welcome to My PWA</h1>
    <p>This app works online and offline!</p>
    <p id="status">Checking connection...</p>
  </div>
  <script>
    // Check online status
    function updateStatus() {
      document.getElementById('status').textContent =
```

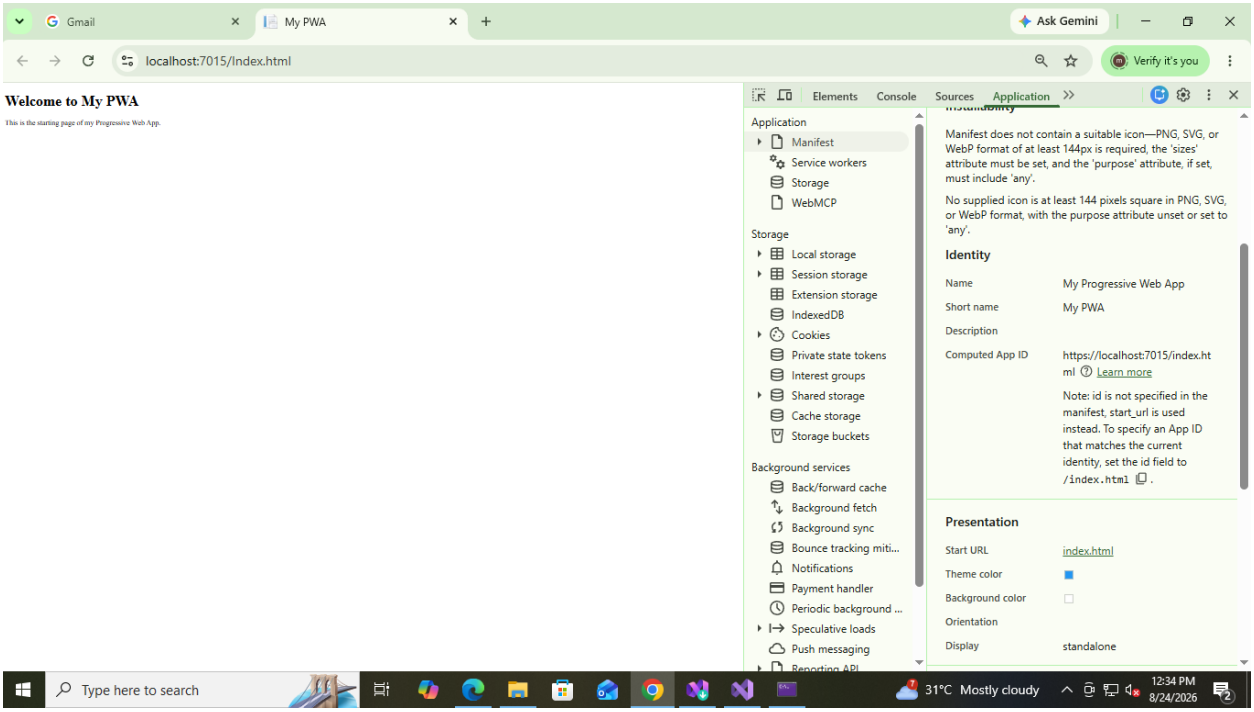
```
      navigator.onLine ? '□ Online' : '□ Offline';
    }
    window.addEventListener('online', updateStatus);
    window.addEventListener('offline', updateStatus);
    updateStatus();
    // Register Service Worker
    if ('serviceWorker' in navigator) {
      window.addEventListener('load', () => {
        navigator.serviceWorker.register('/sw.js').then(reg => {
          console.log('SW registered. Scope:', reg.scope);
        });
      });
    }
  </script>
</body>
</html>
```

Manifest.json

```
{
  "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "description": "A sample PWA application",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1F4E79",
  "lang": "en",
  "orientation": "any",
  "scope": "/",
```

```
 "icons": [
   {
     "src": "icons/icon-72.png",
     "sizes": "72x72",
     "type": "image/png",
     "purpose": "any"
   },
   {
     "src": "icons/icon-192.png",
     "sizes": "192x192",
     "type": "image/png",
     "purpose": "any maskable"
   }urpose": "any maskable"
   },
   {
     "src": "icons/icon-512.png",
     "sizes": "512x512",
     "type": "image/png",
     "p
 ]
}
```

Output:


Gmail
My PWA
Ask Gemini
localhost:7015/Index.html
Verify it's you
Welcome to My PWA
This is the starting page of my Progressive Web App.
Elements
Console
Sources
Application
Application
Manifest
Service workers
Storage
WebMCP
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
Private state tokens
Interest groups
Shared storage
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Bounce tracking miti...
Notifications
Payment handler
Periodic background ...
Speculative loads
Push messaging
Manifest does not contain a suitable icon—PNG, SVG, or WebP format of at least 144px is required, the 'sizes' attribute must be set, and the 'purpose' attribute, if set, must include 'any'.
No supplied icon is at least 144 pixels square in PNG, SVG, or WebP format, with the purpose attribute unset or set to 'any'.
Identity
Name
My Progressive Web App
Short name
My PWA
Description
Computed App ID
https://localhost:7015/index.html
Learn more
Note: id is not specified in the manifest, start_url is used instead. To specify an App ID that matches the current identity, set the id field to /index.html .
Presentation
Start URL
index.html
Theme color
Background color
Orientation
Display
standalone
Type here to search
31°C Mostly cloudy
12:34 PM
8/24/2026

Practical 3b: Simple Web API returning Student data in JSON

AIM : To create a simple ASP.NET Core Web API that returns Student data in JSON format.

Models/Student.cs

```
namespace StudentAPI.Models
{
   public class Student
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public string Course { get; set; }
      public int Age { get; set; }
      public double Percentage { get; set; }
   }
}
```

Controllers/StudentsController.cs

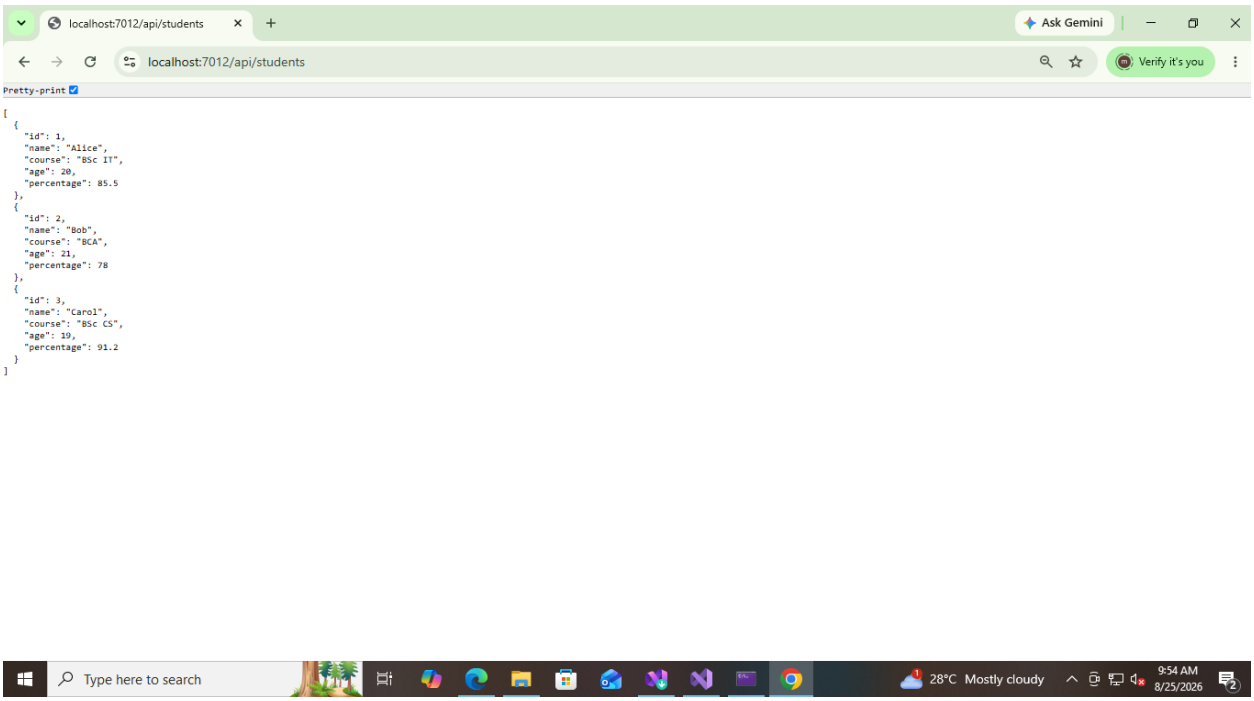
```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

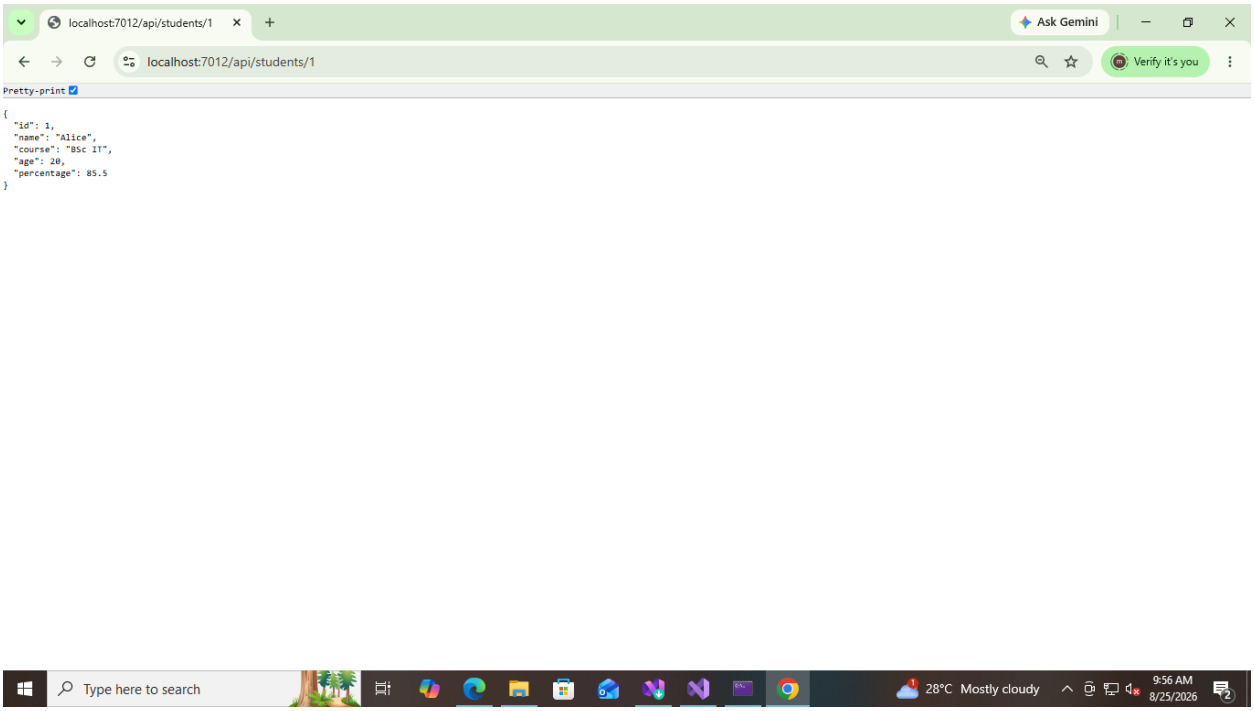
namespace StudentAPI.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   public class StudentsController : ControllerBase
   {
      private static List<Student> students = new List<Student>
      {
         new Student { Id = 1, Name = "Alice", Course = "BSc IT", Age = 20, Percentage = 85.5
},
         new Student { Id = 2, Name = "Bob", Course = "BCA", Age = 21, Percentage = 78.0 },
         new Student { Id = 3, Name = "Carol", Course = "BSc CS", Age = 19, Percentage = 91.2
}
      };
```

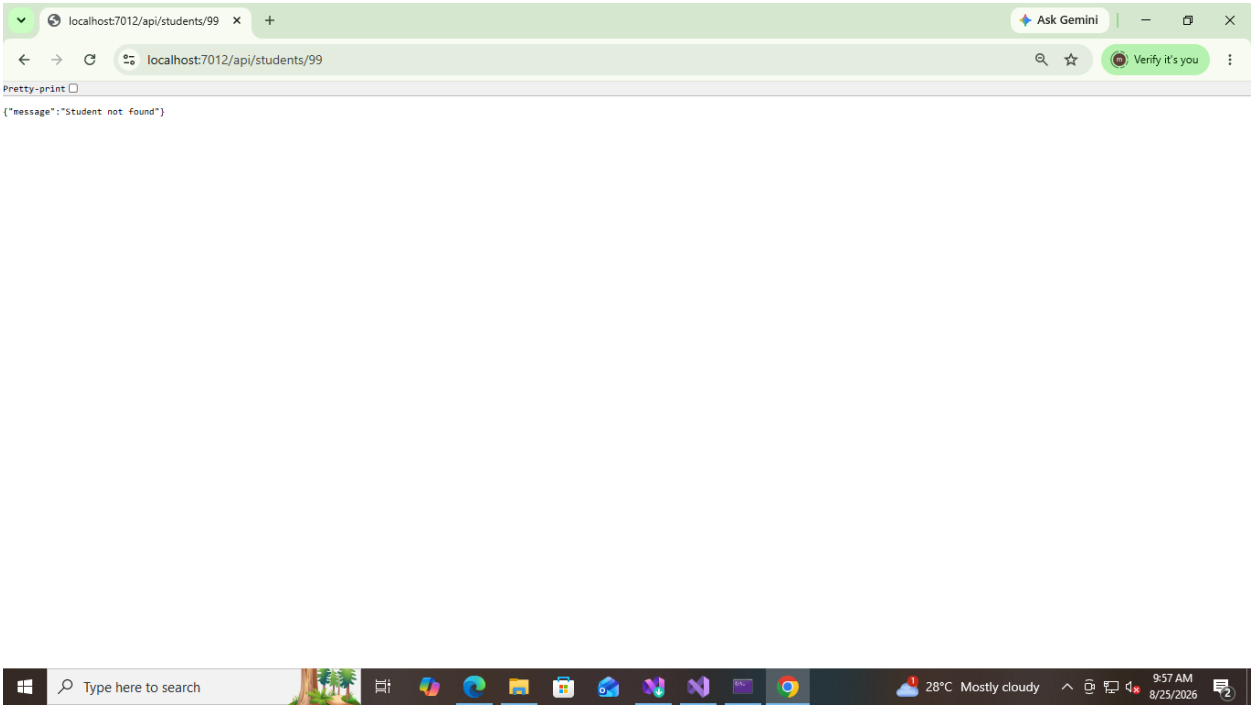
```
// GET: api/students
        [HttpGet]
        public ActionResult<IEnumerable<Student>> GetAll()
            => Ok(students);

        // GET: api/students/1
        [HttpGet("{id}")]
        public ActionResult<Student> GetById(int id)
        {
            var s = students.FirstOrDefault(x => x.Id == id);
            if (s == null) return NotFound(new { message = "Student not found" });
            return Ok(s);
        }
    }
}
```

Output:




localhost:7012/api/students/1
Ask Gemini
localhost:7012/api/students/1
Verify it's you
Pretty-print
{
"id": 1,
"name": "Alice",
"course": "BSc IT",
"age": 20,
"percentage": 85.5
}
Type here to search
28°C Mostly cloudy
9:56 AM
8/25/2026


localhost:7012/api/students/99
Ask Gemini
localhost:7012/api/students/99
Verify it's you
Pretty-print
{"message":"Student not found"}
Type here to search
28°C Mostly cloudy
9:57 AM
8/25/2026

Practical 4a: PWA with Web App Manifest and Push Notifications

AIM: To create a PWA with a complete Web App Manifest file (app name, icons, start URL, display mode)and implement Push Notifications.

manifest.json

```
{
"name": "MyApp - Full Featured PWA",
"short_name": "MyApp",
"description": "A fully featured Progressive Web App",
"start_url": "/index.html?source=pwa",
"scope": "/",
"display": "standalone",
"orientation": "portrait-primary",
"background_color": "#1F4E79",
"theme_color": "#2E75B6",
"lang": "en-US",
"categories": ["education", "utilities"],
"icons": [
{ "src": "/icons/icon-72x72.png", "sizes": "72x72", "type": "image/png" },
{ "src": "/icons/icon-96x96.png", "sizes": "96x96", "type": "image/png" },
{ "src": "/icons/icon-128x128.png", "sizes": "128x128", "type": "image/png" },
{ "src": "/icons/icon-192x192.png", "sizes": "192x192", "type": "image/png",
"purpose": "any maskable" },
{ "src": "/icons/icon-512x512.png", "sizes": "512x512", "type": "image/png",
"purpose": "any maskable" }
],
"shortcuts": [
{
"name": "View Students",
"url": "/students",
"icons": [{ "src": "/icons/students.png", "sizes": "96x96" }]
}
]
}
```

push-notification.js

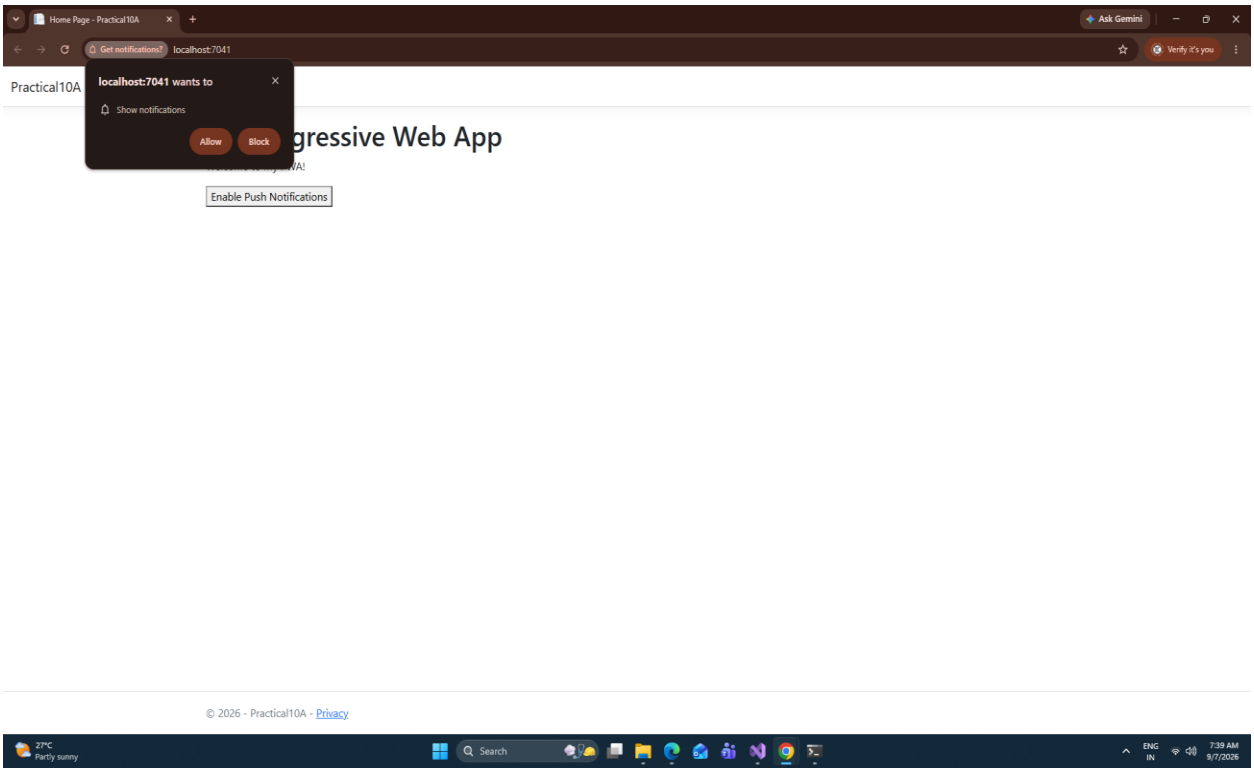
push-notification.js

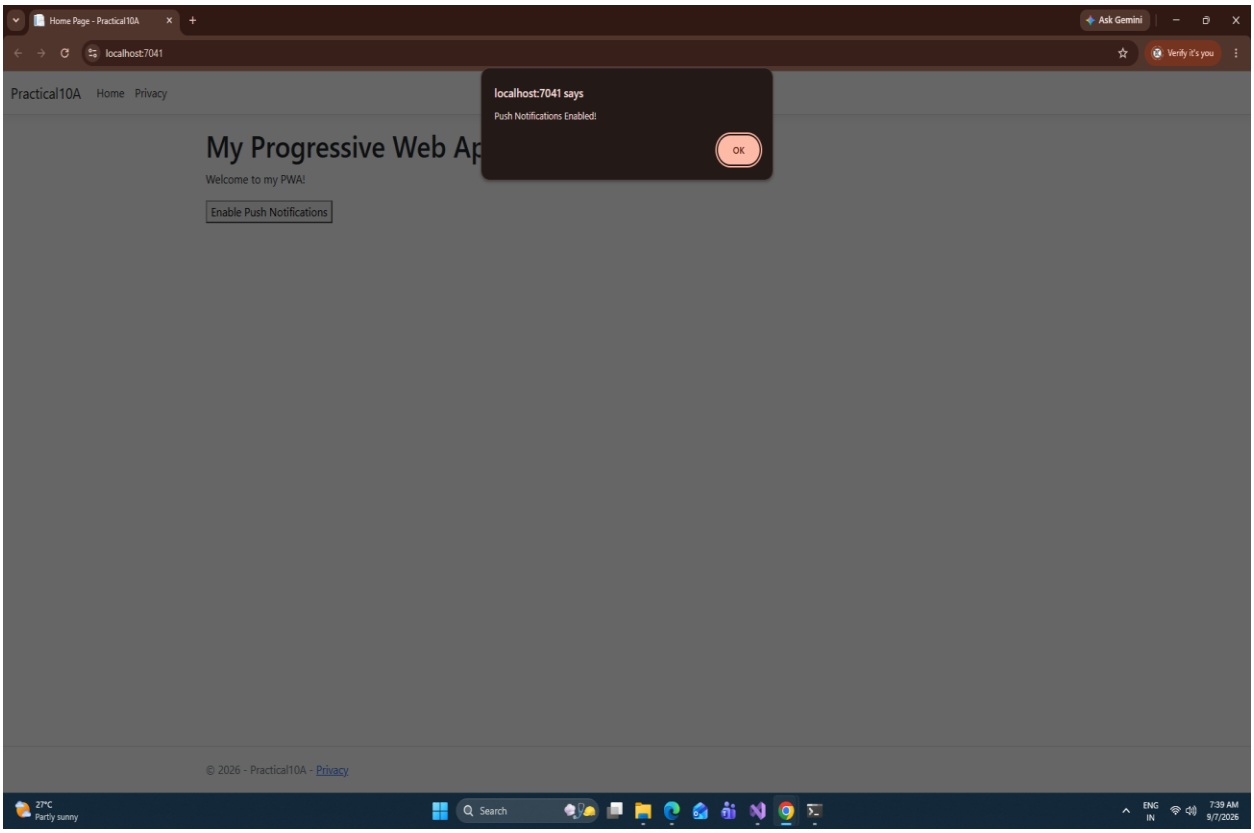
```
// Request notification permission
async function requestPermission() {
```

```
const permission = await Notification.requestPermission();
if (permission === 'granted') {
console.log('Notification permission granted');
subscribeUser();
} else {
console.log('Notification permission denied');
}
}
// Subscribe to push notifications
async function subscribeUser() {
const registration = await navigator.serviceWorker.ready;
const subscription = await registration.pushManager.subscribe({
userVisibleOnly: true,
applicationServerKey: urlBase64ToUint8Array('YOUR_VAPID_PUBLIC_KEY')
});
console.log('Push subscription:', JSON.stringify(subscription));
// Send subscription to server
await fetch('/api/subscribe', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify(subscription)
});
}
// Show local notification demo
function showNotification() {
if (Notification.permission === 'granted') {
new Notification('MyApp Notification', {
body: 'You have a new message!',
icon: '/icons/icon-192x192.png',
badge: '/icons/badge.png',
tag: 'demo-notification'
});
}
}
// Service Worker: push event handler
// (in service-worker.js)
self.addEventListener('push', event => {
const data = event.data ? event.data.json() : {};
const options = {
body: data.body || 'New notification!',
```

```
icon: '/icons/icon-192x192.png',
badge: '/icons/badge.png',
vibrate: [200, 100, 200]
};
event.waitUntil(
self.registration.showNotification(data.title || 'MyApp', options)
);
});
```

Output




Home Page - Practical10A
localhost:7041
Ask Gemini
Practical10A
Home
Privacy
localhost:7041 says
Push Notifications Enabled!
OK
My Progressive Web Ap
Welcome to my PWA!
Enable Push Notifications
© 2026 - Practical10A - Privacy
27°C
Partly sunny
Search
ENG
IN
7:39 AM
9/7/2026

Practical 4b: E-Commerce PWA with Product Listing and Offline Cart

AIM: To develop an E-Commerce Progressive Web Application with product listing and offline cart functionality using Service Worker and local Storage.

PROGRAM CODE

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">

   <meta name="viewport"
       content="width=device-width, initial-scale=1.0">

   <title>ShopPWA - E-Commerce</title>

   <link rel="manifest" href="manifest.json">

   <meta name="theme-color" content="#E74C3C">

   <style>
      * {
         box-sizing: border-box;
      }

      body {
         font-family: Arial, sans-serif;
         margin: 0;
         background: #f5f5f5;
      }

      header {
         background: #E74C3C;
         color: white;
         padding: 15px 20px;
         display: flex;
         justify-content: space-between;
         align-items: center;
```

```
}

.products {
    display: grid;
    grid-template-columns: repeat(
        auto-fit,
        minmax(200px, 1fr)
    );
    gap: 20px;
    padding: 20px;
}

.card {
    background: white;
    border-radius: 8px;
    padding: 15px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    text-align: center;
}

.card img {
    width: 100%;
    height: 150px;
    object-fit: cover;
    border-radius: 4px;
}

.price {
    color: #E74C3C;
    font-size: 1.2em;
    font-weight: bold;
    margin: 8px 0;
}

.btn {
    background: #E74C3C;
    color: white;
    border: none;
    padding: 8px 16px;
    border-radius: 4px;
```

```
  cursor: pointer;
   width: 100%;
}

.btn:hover {
   background: #C0392B;
}

.cart-panel {
   position: fixed;
   right: 0;
   top: 60px;
   background: white;
   width: 300px;
   height: calc(100vh - 60px);
   box-shadow: -2px 0 8px rgba(0, 0, 0, 0.1);
   padding: 20px;
   overflow-y: auto;
   display: none;
}

.cart-panel.open {
   display: block;
}

#cartBadge {
   background: white;
   color: #E74C3C;
   border-radius: 50%;
   padding: 2px 6px;
   font-size: 12px;
   font-weight: bold;
}

#offlineBar {
   background: #FFA500;
   text-align: center;
   padding: 8px;
   display: none;
}
```

```
  </style>
</head>

<body>

  <!-- Offline Message -->
  <div id="offlineBar">
    ⚠ You are offline. Cart saved locally.
  </div>

  <!-- Header -->
  <header>
    <h2>□ ShopPWA</h2>

    <button
      onclick="toggleCart()"
      style="
        background: transparent;
        border: none;
        color: white;
        font-size: 20px;
        cursor: pointer;
      "
    >
      □ Cart
      <span id="cartBadge">0</span>
    </button>
  </header>

  <!-- Product List -->
  <div class="products" id="productList"></div>

  <!-- Cart Panel -->
  <div class="cart-panel" id="cartPanel">

    <h3>Your Cart</h3>

    <div id="cartItems"></div>

    <hr>

    <p>
      Total:
      <strong id="cartTotal">₹0</strong>
    </p>
```

```
    <button class="btn" onclick="checkout()">
      Checkout
    </button>

    <button
      onclick="toggleCart()"
      style="margin-top: 10px; width: 100%;"
    >
      Close
    </button>

  </div>

  <script>

    // Product List
    const products = [
      {
        id: 1,
        name: "Laptop",
        price: 55000,
        emoji: "□"
      },
      {
        id: 2,
        name: "Headphones",
        price: 2500,
        emoji: "□"
      },
      {
        id: 3,
        name: "Smartphone",
        price: 18000,
        emoji: "□"
      },
      {
        id: 4,
        name: "Tablet",
        price: 22000,
        emoji: "□"
      },
      {
        id: 5,
        name: "Keyboard",
        price: 1500,
        emoji: "⌨□
```

```
},
        {
            id: 6,
            name: "Mouse",
            price: 800,
            emoji: "□"
        }
    ];

    // Load cart from Local Storage
    let cart = JSON.parse(
        localStorage.getItem("pwa-cart") || "[]"
    );

    // Display Products
    function renderProducts() {

        document.getElementById("productList").innerHTML =
            products.map(p => `

                <div class="card">

                    <div style="font-size: 80px">
                        ${p.emoji}
                    </div>

                    <h3>${p.name}</h3>

                    <div class="price">
                        ₹${p.price.toLocaleString()}
                    </div>

                    <button
                        class="btn"
                        onclick="addToCart(${p.id})"
                    >
                        Add to Cart
                    </button>

                </div>

            `).join("");
    }

    // Add Product to Cart
    function addToCart(productId)
```

```
        const product = products.find(
            p => p.id === productId
        );

        const existing = cart.find(
            item => item.id === productId
        );

        if (existing) {
            existing.qty++;
        } else {
            cart.push({
                ...product,
                qty: 1
            });
        }

        saveCart();
        renderCart();

        alert(`${product.name} added to cart!`);
    }

    // Save Cart
    function saveCart() {

        localStorage.setItem(
            "pwa-cart",
            JSON.stringify(cart)
        );
    }

    // Display Cart
    function renderCart() {

        const total = cart.reduce(
            (sum, item) =>
                sum + item.price * item.qty,
            0
        );

        // Cart Badge
        document.getElementById("cartBadge").textContent =
            cart.reduce(
                (sum, item) => sum + item.qty,
);
```

```
        // Cart Total
        document.getElementById("cartTotal").textContent =
          `₹${total.toLocaleString()}`;

        // Cart Items
        document.getElementById("cartItems").innerHTML =
          cart.map(item => `

            <div
              style="
                display: flex;
                justify-content: space-between;
                margin: 8px 0;
              "
            >

              <span>
                ${item.emoji}
                ${item.name} x${item.qty}
              </span>

              <span>
                ₹${(
                  item.price * item.qty
                ).toLocaleString()}
              </span>

            </div>

          `).join("") || "<p>Cart is empty</p>";
      }

      // Open / Close Cart
      function toggleCart() {

        document
          .getElementById("cartPanel")
          .classList.toggle("open");
      }

      // Checkout
      function checkout() {
 if (!navigator.onLine)
          alert(
            "You are offline. Order will be placed when you reconnect."
```

```
            );

        } else {

            alert(
                "Order placed successfully! Thank you!"
            );

            cart = [];

            saveCart();
            renderCart();
        }
    }

    // Offline Event
    window.addEventListener("offline", () => {

        document.getElementById(
            "offlineBar"
        ).style.display = "block";

    });

    // Online Event
    window.addEventListener("online", () => {

        document.getElementById(
            "offlineBar"
        ).style.display = "none";

    });

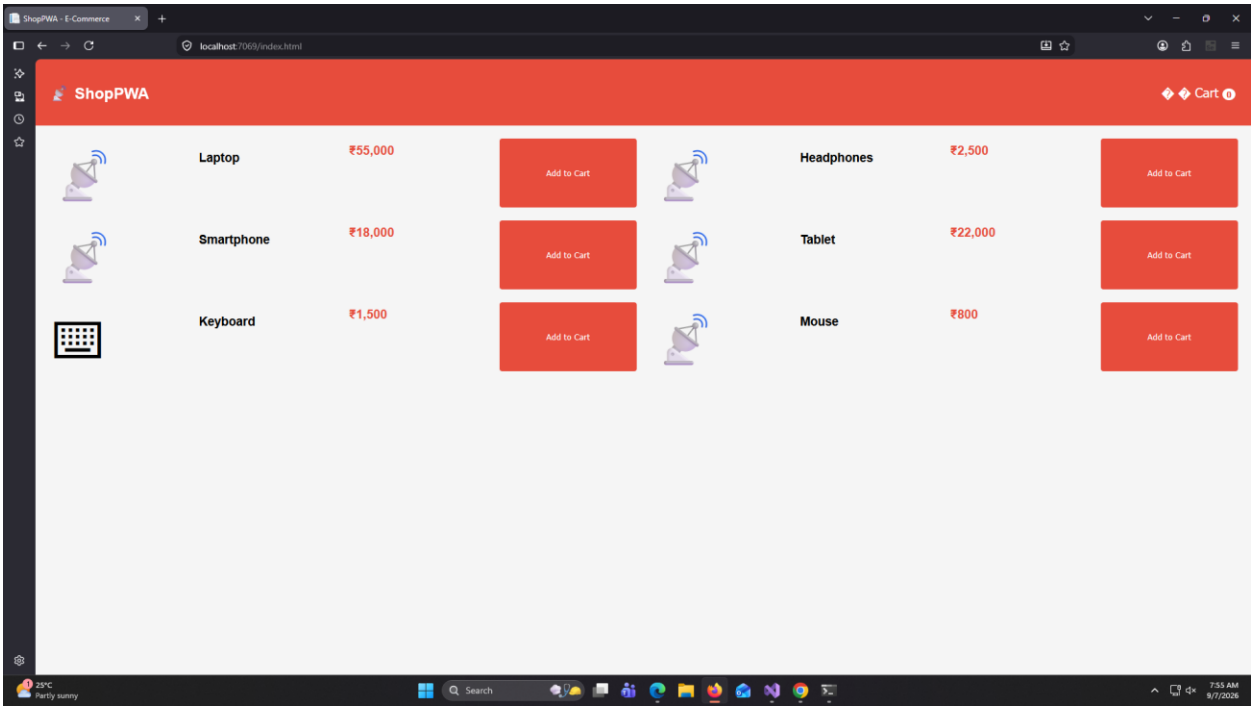
    // Initial Display
    renderProducts();
    renderCart();

    // Register Service Worker
    if ("serviceWorker" in navigator) {

        navigator.serviceWorker.register("/sw.js");

  </script>
</body>
</html>
```

Output


ShopPWA - E-Commerce
localhost:7069/index.html
ShopPWA
Cart 0
Laptop
₹55,000
Add to Cart
Headphones
₹2,500
Add to Cart
Smartphone
₹18,000
Add to Cart
Tablet
₹22,000
Add to Cart
Keyboard
₹1,500
Add to Cart
Mouse
₹800
Add to Cart
25°C
Partly sunny
Search
7:55 AM
9/7/2026